TECHNICAL SPECIFICATION

V4.0 · 2026-04-29

LSIE-MLF

Master Technical Specification

Windows Desktop Runtime, Local State, IPC, and Cloud Control Plane

Live Stream Inference Engine - Machine Learning Framework

CONTENT VERSION	SCHEMA VERSION	CLASSIFICATION	HARDENING BASELINE
v4.0	v4.0	CONFIDENTIAL	Gate 0

CONTENTS

Table of contents

§0 **Document control and canonical terminology** 3
- §0.1 Document purpose 3
- §0.2 Normative language 3
- §0.3 Canonical terminology registry 3
- §0.4 Version history 5
- §0.5 Gate 0 replay invariants 6

§1 **System philosophy and operational scope** 6

§2 **Deterministic system data flow pipeline** 7

§3 **Codebase architecture and directory mapping** 9
- §3.1 Directory hierarchy 9
- §3.2 Build context rules 10

§4 **Core architecture modules** 11
- §4.A Hardware & transport 11
- §4.B Ground truth ingestion 13
- §4.C Orchestration & synchronization 14
- §4.D Multimodal delivery validation and response inference 15
- §4.E Experimentation & analytics 17
- §4.F Cloud synchronization 18

§5 **Data governance and security** 19
- §5.4 Desktop local state 19
- §5.1 Ephemeral vault specification 20
- §5.2 Data classification 20

§6 **Interface contracts and payload schemas** 21
- §6.1–6.4 Schema registry 22

§7 **Mathematical specifications** 23
- §7A AU12 facial action unit geometry 23
- §7B Thompson sampling reward pipeline 26
- §7C Physiological co-modulation analytics 30
- §7D Observational acoustic analytics 33
- §7E Event→Outcome attribution policy 36

§8 **Deterministic delivery validation and response inference specification** 41
- §8.2.1 Inference parameter constraints 42
- §8.1 Deterministic semantic scorer registry 42

§8.3 Expected output schema . . . 43
§8.4 System prompt (canonical) . . . 43

§9 Desktop runtime topology 43
§9.1 Desktop process graph . . . 44
§9.2 Desktop IPC contract . . . 45
§9.3 Native subprocess supervision . . . 45
§9.4 Runtime base . . . 45
§9.5 Startup sequence . . . 45
§9.6 Tier-2 platform design intent . . . 46

§10 Dependency and runtime matrix 46
§10.1 System requirements . . . 46
§10.2 Python package pinning . . . 47

§11 Variable extraction matrix 48

§12 Error handling and recovery specification 58
Hardware device loss . . . 58
Worker process crash . . . 59
Temporal drift fault . . . 59
SharedMemory leak . . . 59
ML import failure . . . 59
Pascal speech incompatibility . . . 59
Acoustic data quality . . . 60
SQLite Busy/Crash recovery . . . 60
Reward bounds fault . . . 60
Network disconnection . . . 60
Cloud duplicate . . . 60
Privacy perimeter violation . . . 60
Operator UI exit . . . 61

§13 Autonomous implementation audit checklist 61

§14 Cloud control plane and telemetry sync 63
§14.1 Cloud route registry . . . 63

SECTION 0

Document control and canonical terminology

§0.1 Document purpose

LSIE-MLF v4.0 is an implementation contract for the desktop runtime, local state layer, IPC graph, cloud control plane, and mathematical pipeline. The active runtime vocabulary is `ProcessGraph`, IPC queues/shared-memory blocks, SQLite local state, and cloud outbox synchronization; legacy container names remain only for retained server/cloud paths and historical traceability. The embedded JSON remains the machine authority, while the rendered document presents the same contract in navigable prose.

This document is the single authoritative v4.0 specification for the Live Stream Inference Engine - Machine Learning Framework (LSIE-MLF). A developer or autonomous AI agent is required to build the complete v4.0 desktop runtime, local state layer, IPC graph, cloud control plane, and mathematical pipeline using only this document. Every implementation-relevant entry restates the governing rule locally so the artifact remains self-sufficient.

§0.2 Normative language

Throughout this specification, the following terms carry precise normative meaning:

TERM	MEANING
MUST / SHALL / IS REQUIRED TO	The requirement is absolute and non-negotiable.
MUST NOT / SHALL NOT	The prohibition is absolute.
IS DEFINED AS	The term has a single, unambiguous meaning as stated.

§0.3 Canonical terminology registry

Active v4.0 implementation prose must use services.desktop_app process and transport terminology. Container-era names are retained only for explicit legacy/server/cloud scope or historical traceability.

CANONICAL NAME	RETIRED SYNONYMS	DEFINITION
ui_api_shell		Active v4 desktop process that owns the PySide6 operator shell and loopback FastAPI surface backed by SQLite local state.
capture_supervisor		Active v4 desktop process that supervises ADB, scrcpy, and FFmpeg native subprocesses through the OS adapter and process cleanup policy.
module_c_-orchestrator		Active v4 desktop process that synchronizes capture events, applies drift correction, derives AU12 and physiological context, assembles segments, and produces IPC work.
gpu_ml_worker		Active v4 desktop child process that owns GPU ML imports, speech transcription, delivery validation, response inference, and observational acoustic analytics.
analytics_state_-worker		Active v4 desktop process that is the sole writer to SQLite local state and owns reward, posterior, attribution, and upload-outbox records.
cloud_sync_worker		Active v4 desktop process that drains the SQLite upload outbox, refreshes OAuth tokens, uploads derived telemetry, and fetches signed experiment bundles.
ProcessGraph		The active v4 runtime graph containing exactly ui_api_shell, capture_supervisor, module_c_orchestrator, gpu_ml_worker, analytics_state_worker, and cloud_sync_worker.
IPC queues/shared-memory blocks		The active v4 local transport: multiprocessing.Queue control messages carry metadata and SharedMemory names for 30-second PCM windows.
SQLite local state		The active v4 local derived-state store in WAL mode, written only by analytics_state_worker and read by approved local processes.
cloud outbox uploads		Durable SQLite pending_uploads records drained by cloud_sync_worker over HTTPS to the cloud control plane.
InferenceHandoffPay-load		The Pydantic and JSON Schema contract for Module C to Module D and Module D to Module E exchange.
BanditDecisionSnap-shot		The deterministic pre-update Thompson Sampling decision record attached to each eligible segment.
PhysiologicalChun-kEvent		A normalized wearable telemetry chunk record used only after provider hydration and schema validation.
Physiological Context		A per-subject derived physiological snapshot attached to eligible segments without persisting raw provider bodies.
Co-Modulation Index		A derived Pearson correlation over aligned valid RMSSD sequences, persisted as analytics rather than raw physiology.
AttributionEvent		The canonical derived event record for stimulus-linked interactions eligible for attribution analysis.

CANONICAL NAME	RETIRED SYNONYMS	DEFINITION
OutcomeEvent		The canonical derived delayed outcome record, such as creator_follow, used by attribution analysis.
EventOutcomeLink		The deterministic linkage record between an AttributionEvent and an eligible OutcomeEvent.
AttributionScore		The method-specific derived attribution or observational score record.
bounded_reason_code		A bounded semantic or delivery-validation reason-code value; free-form rationales must not be persisted.
soft_reward_candidate		An observational semantic attribution candidate that does not modulate section 7B reward or posterior updates.

§0.4 Version history

VERSION	DATE	DESCRIPTION
v1.0	2026-03-12	Initial unified specification.
v2.0	2026-03-12	Production-hardened. Applied 15-rule deterministic audit: canonical terminology, module contracts, data-flow pipeline, dependency pinning, AU12 Python fix, IPC lifecycle, temporal sync formulas, Ephemeral Vault hardening, Docker topology expansion, LLM determinism constraints, error-handling matrices, variable matrix expansion, and autonomous implementation audit.
v3.0	2026-04-01	Mathematical recipe (Thompson Sampling, continuous reward, fractional Beta-Bernoulli update), E2E gap fixes, hardening baseline, 6 spec amendments incorporated (cuDNN 8, Ubuntu 24.04, orchestrator container, scrcpy v3.3.4, audio chunk 1/30s, INT8 locked), 63 new tests, reward pipeline formalization.
v3.1	2026-04-15	Physiological co-modulation extension. Added Oura Ring webhook ingestion (section 4.B.2), Orchestrator physiological state buffer (section 4.C.4), Module E co-modulation analytics (section 4.E.2), PhysiologicalChunkEvent and PhysiologicalContext schemas (section 6.2, section 6.3), RMSSD co-modulation math (section 7C), physiological data governance, failure matrix entries, variable matrix entries, and audit checklist items. Section 7B Thompson Sampling reward pipeline explicitly unchanged.
v3.2	2026-04-17	Physiological ingestion model corrected. Oura webhook handling no longer assumes provider-delivered scalar 5-minute RMSSD payloads. Webhooks are treated as change notifications that trigger OAuth2-backed hydration, normalized physiological chunk transport, rolling RMSSD derivation in Module C, validity gating, and 1-minute co-modulation resampling. Operator dashboard stack migrated from Streamlit to dual PySide6 host surfaces: a retained single-file developer debug tool and a modular production operator console. Section 7B reward pipeline remains unchanged.
v3.3	2026-04-23	Observational acoustic analytics extension. Added stimulus-windowed acoustic analysis in Module D, new mathematical recipe (section 7D), canonical Module D -> Module E payload fields, Persistent Store schema-oriented acoustic field additions, deterministic acoustic data-quality handling, and new audit checklist items. Section 7B Thompson Sampling reward pipeline remains unchanged.

VERSION	DATE	DESCRIPTION
v3.4	2026-04-24	Attribution analytics extension. Added Module E Attribution Analytics (section 4.E.3), stable segment identity and BanditDecisionSnapshot carriage in InferenceHandoffPayload (section 6.1), attribution schemas (section 6.4), Event->Outcome Attribution Policy (section 7E), deterministic semantic scorer registry with local cross_encoder primary and optional llm_gray_band fallback (section 8), bounded semantic reason codes, variable matrix additions, and new audit controls. Section 7B Thompson Sampling reward pipeline remains unchanged.
v4.0	2026-04-29	Desktop runtime pivot. Replaces Docker Compose runtime with Windows Tier 1 six-process desktop graph; introduces python-build-standalone plus uv runtime hydration, Queue plus SharedMemory IPC, SQLite WAL local state, cloud control plane, deterministic offline arm selection, posterior delta sync, Turing production GPU floor, Pascal CPU-speech developer override, and Gate 0 replay invariants. Section 7B reward mathematics remain unchanged.

§0.5 Gate 0 replay invariants

Gate 0 defines the replay checks that protect the v4 desktop runtime before implementation work proceeds. The checks cover deterministic segment identity, 30-second windowing, ADB drift behavior, §7B reward invariance, Thompson Sampling posterior updates, immutable bandit decision snapshots, schema-validated handoffs, privacy boundaries, stimulus-response fixture coverage, and the rule that `response_inference` remains observational. These checks establish which behaviors must remain reproducible across code changes.

SECTION 1

System philosophy and operational scope

LSIE-MLF v4.0 is a privacy-preserving desktop runtime first and a cloud-synchronized analytics system second. The local ProcessGraph observes stimulus delivery and bounded subject response without persisting raw audio, decoded frames, voiceprints, provider payload bodies, or free-form semantic rationales. Raw media stays transient, process ownership is explicit, inter-process handoffs are schema-validated, and cloud sync carries only derived records that can be replayed and audited.

LSIE-MLF v4.0 measures stimulus delivery and observable subject response over USB-tethered multimodal streams - PCM audio, H.264 video, and chat metadata - inside a Windows-native desktop process graph. For each encounter, the system records one selected stimulus attempt, validates whether that stimulus was delivered using observable evidence, observes synchronized multimodal response evidence, computes reward under section 7B, and updates experiment state.

The architecture remains schema-first and deterministic. USB media transport, temporal synchronization, message passing, local state, cloud sync, delivery validation, response inference, and ML

inference must behave predictably across independent audio, video, chat, physiological, and cloud clocks. The system must not treat unobserved internal perception as ground truth.

The v4.0 runtime uses six Windows processes, multiprocessing.Queue control messages, SharedMemory audio transport, SQLite WAL local state, and HTTPS cloud synchronization. Stimulus-response semantics do not alter that runtime topology.

The system applies strict data minimization. Raw biometric media is processed transiently and must not persist locally or in cloud storage; persistent state is limited to derived, normalized outputs under the desktop privacy and cloud perimeter rules in section 5.

SECTION 2

Deterministic system data flow pipeline

A segment moves from mobile capture through orchestration, ML inference, local analytics, and cloud synchronization under explicit ownership at every boundary. Each handoff defines the process that owns the data, the representation allowed to cross the boundary, and the deterministic failure state. Raw media and unbounded rationale text stay out of SQLite and cloud transport; bounded handoffs, analytics records, and posterior deltas are the durable surfaces.

The Windows-native LSIE-MLF desktop runtime moves data through named process-to-process transitions, with each stage title stating the operational role of that handoff. In v4.0 stimulus-response processing, each selected arm is represented as a `stimulus_attempt` anchored at `trigger_time_utc`. The `delivery_validation` fields determine whether the stimulus satisfied `expected_stimulus_rule`; `subject_response` evidence is evaluated only inside `response_inference`, and delayed platform outcomes remain separate `outcome_event` records. The preserved §7B reward contract remains invariant across the desktop transport.

§2.1 MOBILE HARDWARE → CAPTURE_SUPERVISOR

Tethered mobile hardware becomes runtime data only inside `capture_supervisor`. `ADB`, `scrcpy`, and `FFmpeg` are supervised as native children, raw PCM and H.264 remain transient, and device health is converted into bounded capture status. Decoded frames and raw audio stay inside the local process boundary; failures surface through supervised shutdown or reconnection rather than persistent media files.

§2.2 CAPTURE_SUPERVISOR → MODULE_C_ORCHESTRATOR

Supervised capture state reaches `module_c_orchestrator` as control information, not as a general media store. Drift offsets, process health, and pipe endpoints describe how capture is behaving while media remains in memory or process-owned streams. Drift and capture health are recoverable state; raw capture data is transient working material.

§2.3 MEDIA SYNCHRONIZATION AND SEGMENT ASSEMBLY

Segment assembly converts continuous capture into one deterministic 30-second analytical unit. The `module_c_orchestrator` process aligns timestamps, extracts AU12-derived facial telemetry, attaches physiological context when available, and selects at most one stimulus attempt for the segment. Missing AU12 or physiology produces explicit null or false-validity outputs instead of broken segment identity or invented reward evidence.

§2.4 EVENT NORMALIZATION, ACTIVE-ARM POLICY, AND STIMULUS ATTEMPT SELECTION

Event normalization connects bounded external evidence to the active desktop experiment policy. Module B and the signed ExperimentBundle provide structured platform events, stimulus attempts, expected delivery rules, and expected response rules; Module C decides which event belongs to a segment. Bounded stimulus payloads are distinct from raw platform content, especially when new modalities or response evidence are introduced.

§2.5 MODULE_C_ORCHESTRATOR → GPU_ML_WORKER

The ML handoff is the main process boundary between orchestration and heavy inference. The `module_c_orchestrator` process sends a schema-validated InferenceHandoffPayload plus a shared-memory reference for the PCM window, and `gpu_ml_worker` attaches with consumer semantics rather than taking ownership of persistence. Checksum, zeroization, and orphan cleanup matter here because large transient audio crosses the boundary without being copied into SQLite or cloud payloads.

§2.6 DELIVERY VALIDATION AND RESPONSE INFERENCE

Delivery validation and response inference are produced together but interpreted differently. The `gpu_ml_worker` process emits bounded top-level delivery-validation fields for the §7B reward gate and a separate `response_inference` object for observational subject-response analytics. Acoustic insufficiency, ambiguous language, or missing transcript evidence can produce deterministic nulls or reason codes; response inference does not update Thompson Sampling posteriors in v4.0.

§2.7 ANALYTICS_STATE_WORKER → SQLITE WAL AND CLOUD_SYNC_WORKER

Derived observations become local state through `analytics_state_worker`, the sole SQLite writer. Reward results, posterior updates, attribution records, physiological summaries, acoustic summaries, response-inference analytics, and `pending` upload rows all pass through this process. Cloud failure affects only the outbox drain; local posterior state and replayable analytics remain authoritative while offline.

§2.8 CLOUD_SYNC_WORKER ↔ CLOUD CONTROL PLANE

Cloud synchronization exports bounded derived records and imports signed experiment bundles. The `cloud_sync_worker` process uses OAuth/PKCE, local credential storage, idempotency keys, and schema-on-write privacy rejection so cloud availability can lag behind local operation safely. Outbox retries must not duplicate posterior deltas, and no raw media, raw provider body, or free-form rationale crosses the cloud perimeter.

SECTION 3

Codebase architecture and directory mapping

The repository separates specification intent from runtime surfaces. Desktop processes, retained server/cloud services, shared schemas, ML and mathematical utilities, scripts, tests, and generated artifacts each carry a different authority boundary. The shared `packages/schemas` tree defines typed contracts, `packages/ml_core` holds reusable ML and mathematical utilities, `services/desktop_app` owns the active desktop runtime, and `docs/artifacts` contains generated spec payloads rather than application logic.

The repository remains a segmented monorepo, but v4.0 replaces the normative Docker Compose runtime with a Windows-native desktop application surface.

The six-process graph isolates UI/API, capture supervision, orchestration, GPU ML imports, analytics state writes, and cloud synchronization.

Only `gpu_ml_worker` may import torch, ctranslate2, mediapipe, and faster_whisper as a combined heavy-ML process boundary; `module_c_orchestrator` may import MediaPipe only for AU12/Face Mesh where required.

§3.1 Directory hierarchy

PATH	PURPOSE
`/packages/`	Shared Python packages used across runtime surfaces. `packages/schemas/` owns Pydantic/JSON Schema contracts; `packages/ml_core/` owns reusable ML, reward, attribution, encryption, and mathematical utilities.

PATH	PURPOSE
`/services/desktop_app/`	Active v4 desktop runtime launched by `python -m services.desktop_app`. Contains process graph orchestration, IPC/shared-memory helpers, SQLite local state, cloud outbox clients, and desktop process entrypoints.
`/services/desktop_launcher/`	Windows launcher and install-management shell. It prepares the embedded Python runtime, verifies dependencies, runs repair/reinstall workflows, and should remain separate from analytical runtime logic.
`/services/operator_console/`	PySide6 operator console surface for live-session, experiment, physiology, health, and session views. It consumes API/viewmodel state rather than opening direct database or broker connections.
`/services/api/`	Retained FastAPI server/cloud API surface for legacy/server routes, operator aggregate APIs, physiology ingress, and analytical read endpoints where explicitly scoped by the spec.
`/services/worker/`	Retained worker/orchestration implementation for server/cloud and deferred integration paths, including inference tasks, analytics persistence, enrichment, ground-truth ingestion, and vault cron code.
`/services/cloud_api/`	Cloud control-plane implementation for OAuth/PKCE, telemetry ingest, signed experiment bundles, session records, and posterior-delta application.
`/scripts/`	Developer, audit, fixture-generation, schema-consistency, operator tooling, and specification validation scripts. These are operational tools, not long-running runtime processes unless a specific entrypoint says so.
`/tests/`	Unit, integration, static, and Gate 0 replay tests. Test fixtures under this tree document expected behavior for process topology, schemas, analytics, privacy boundaries, and deterministic replay.
`/docs/`	Signed specification references, amendment registries, operational process documents, and generated artifacts. `docs/artifacts/` contains draft/render inputs and should be treated as spec-output material rather than application code.
`/data/`	SQL bootstrap and retained persistence assets used by server/cloud surfaces and audit comparisons. Raw biometric media should not be added here.
`/build/`	Packaging configuration such as PyInstaller specs and launcher build assets.
`/requirements/`	Dependency lock and install-target support where present; package pins remain governed by `pyproject.toml` and `uv.lock`.
`/pyproject.toml and /uv.lock`	Authoritative Python dependency declarations and frozen resolution. Base dependencies, extras, and ML-heavy packages must stay scoped to their intended install targets.
`/.github/`	Repository automation and Dependabot process configuration. Dependabot handling follows `.github/DEPENDABOT_PROCESS.md` and is not bundled into feature-review work.

§3.2 Build context rules

v4.0 builds a signed Windows desktop launcher and hydrates the ML runtime with python-build-standalone plus `uv`.lock. `PyInstaller` one-folder mode may wrap the launcher shell only; heavy ML wheels must not be re-extracted on every app launch.

SECTION 4

Core architecture modules

Modules A through F divide the runtime by ownership rather than by implementation convenience. Capture supervision, event normalization, orchestration, ML interpretation, analytics state, and cloud synchronization each have a bounded interface and a distinct failure mode. The module map is the safest way to decide where a new responsibility belongs before touching code.

The modules defined below form the operational spine of LSIE-MLF v4.0. Module identities remain stable for indexing, and active contracts are scoped to the Windows-native six-process desktop graph. Historical or deferred integrations are informative only unless a module contract explicitly marks them as active.

§4.A Hardware & transport

Module A is the hardware-facing edge of the desktop runtime. The `capture_supervisor` process supervises `ADB`, `scrcpy`, and `FFmpeg`, converts device streams into local transient media flows, and reports bounded capture and drift state to the rest of the ProcessGraph. Raw audio and video are visible only while capturing and are never persisted as biometric media or exposed as decoded frames across process boundaries.

Purpose. Extract low-latency audio and video streams from tethered mobile hardware and transport them into the local desktop process graph without REST, broker, or container serialization overhead.

Technology stack. scrcpy v3.3.4, ADB server, FFmpeg, pywin32 Job Objects, multiprocessing process supervision

§4.A.1 EXECUTION DETAILS

Module A begins by proving the local capture environment is usable. `ADB` must see the device, `scrcpy` and `FFmpeg` must be available, and native children must be supervised as a group. Retry, cleanup, and device-loss behavior stay inside `capture_supervisor` so downstream modules receive bounded status rather than half-open subprocesses or stale media handles.

IPC PIPE LIFECYCLE SPECIFICATION

1. **Directory creation:** The Capture Container initializes the shared IPC mount by running `mkdir -p /tmp/ipc` before any scrcpy process is started.
2. **Pipe cleanup and creation:** `setup_pipes()` removes stale FIFO files with `rm -f` and recreates both `/tmp/ipc/audio_stream.raw` and `/tmp/ipc/video_stream.mkv` with `mkfifo`.
3. **Audio fd 3 shield:** The shell opens `exec 3<> $AUDIO_PIPE` so the audio pipe stays open across reader or scrcpy restarts and the writer is decoupled from consumer timing.
4. **Video pipe left unshielded:** No extra descriptor is opened for `/tmp/ipc/video_stream.mkv`; if the PyAV consumer drops the pipe, the video scrcpy process exits and the next launch emits a fresh MKV header.
5. **Device wait gate:** `wait_for_device()` polls `adb devices` every 2 seconds for up to 60 seconds and terminates the container if no Android device reports a ready device state.
6. **ADB tunnel cleanup:** Each capture loop starts by running `adb forward --remove-all` and `adb reverse --remove-all` so stale tunnels cannot collide with the audio and video port ranges.
7. **Audio stream launch:** The audio scrcpy instance launches first and writes directly to `/tmp/ipc/audio_stream.raw` with `--record` while binding port range 27100:27199 through `$HOST_IP`.
8. **Staggered video launch:** After a 4-second delay, the video scrcpy instance launches and writes H.264 in an MKV streaming container directly to `/tmp/ipc/video_stream.mkv` on port range 27200:27299.
9. **Monitoring and restart:** The container waits on both scrcpy PIDs with `wait -n`; when either process exits, both children are cleaned up, the device is rechecked, and the dual-stream capture loop restarts.
10. **Cross-launch cleanup:** On the next container start, `setup_pipes()` again runs `rm -f` on both FIFO paths before recreating them, preventing stale shared-volume pipe files from surviving across launches.

CONTRACT REFERENCE

BOUNDARY	CONTRACT REFERENCE
INPUTS	USB-connected Android device producing raw PCM audio and encoded H.264 video through supervised native subprocesses.
OUTPUTS	CaptureStatus, DriftOffset, NativeProcessHealth events, and local capture streams consumed by module_c_orchestrator inside the desktop process graph. Raw media remains transient and does not persist.
OWNER	capture_supervisor
RUNTIME BOUNDARY	v4.0 desktop process graph

Failure behavior is summarized in this module's narrative and in the error-handling matrix; detailed failure-mode items remain in the embedded machine-readable artifact.

§4.B Ground truth ingestion

Module B is the normalization layer for bounded external evidence. Platform events, stimulus-trigger records, and optional physiological samples become structured inputs for Module C only after malformed, duplicate, unbounded, or privacy-unsafe data has been rejected.

Purpose. Ingest real-time contextual metadata, chat streams, user interactions, stimulus-trigger evidence, and normalized physiological sample events for downstream desktop assembly.

Technology stack. TikTokLive-compatible event client, Pydantic event schemas, OAuth2 provider client, in-memory desktop event queues

§4.B.1 EXECUTION DETAILS

Module B treats external event streams as untrusted inputs until they have been normalized. Platform identifiers, stimulus attempts, bounded payload evidence, and timing fields become useful only after idempotency, shape, and privacy checks pass. New event types need bounded fields for Module C and deterministic malformed/duplicate behavior.

§4.B.2 PHYSIOLOGICAL INGESTION ADAPTER

Physiological ingestion contributes derived samples to the desktop physiological buffer without authorizing raw provider-response persistence. Provider-specific material becomes normalized physiological sample events with subject role, source timing, and validity information. Stale, absent, and no-RMSSD states remain distinct because co-modulation analytics depend on knowing why a physiological value is null.

CONTRACT REFERENCE

BOUNDARY	CONTRACT REFERENCE
INPUTS	Platform event records, chat/interaction metadata, stimulus-trigger evidence, and provider-derived physiological sample DTOs from configured desktop adapters.
OUTPUTS	Validated event DTOs with uniqueId, event_type, timestamp_utc, stimulus-trigger metadata, and optional PhysiologicalChunkEvent-compatible sample records for Module C buffering.
OWNER	module_c_orchestrator
RUNTIME BOUNDARY	v4.0 desktop process graph

Failure behavior is summarized in this module's narrative and in the error-handling matrix; detailed failure-mode items remain in the embedded machine-readable artifact.

§4.C Orchestration & synchronization

Module C is the deterministic segment builder. The `module_c_orchestrator` process aligns media and event time, applies drift correction, extracts AU12 telemetry, attaches physiological context, captures the pre-update bandit decision snapshot, and emits the canonical 30-second handoff for ML inference.

Purpose. Assemble deterministic 30-second desktop segments by applying drift correction, decoding local video within the process boundary, extracting AU12, attaching physiological context, capturing the bandit decision snapshot, and producing DesktopIPCControlMessage records.

Technology stack. FFmpeg persistent subprocess, PyAV, MediaPipe Face Mesh, multiprocessing.Queue, multiprocessing.shared_memory.SharedMemory, Pydantic schemas

§4.C.1 TEMPORAL DRIFT CORRECTION

Drift correction gives segment windows stable time semantics when device and host clocks diverge. Module C uses `ADB`-derived offsets while they are healthy, freezes after repeated failures, and resets after the defined timeout so a bad drift source does not continuously corrupt segment identity. Window bounds, stimulus trigger time, and reward windows remain explainable from recorded offset behavior.

§4.C.2 VIDEO TELEMETRY AND SEGMENT ASSEMBLY

Video processing in Module C derives bounded facial telemetry rather than moving decoded frames around the runtime. Frames are decoded inside `module_c_orchestrator`, AU12 observations are extracted, and the segment record carries derived time series plus identity metadata. Visual analytics outputs must remain bounded observations suitable for validation or replay, never persisted frame buffers.

§4.C.3 AUDIO RESAMPLING

Audio resampling prepares one canonical PCM representation for the ML worker. Module C converts transient capture audio into 16 kHz mono PCM, writes the 30-second window into shared memory, and sends only the control envelope plus `checksum` across the queue. Deterministic resampling keeps transcription, acoustic analysis, and replay fixtures aligned on the same window shape and byte accounting.

§4.C.4 PHYSIOLOGICAL STATE BUFFER

The physiological buffer turns asynchronous wearable or provider samples into per-segment context. It maintains rolling state by `subject_role`, marks freshness and validity explicitly, and allows stale

or absent context to travel as bounded metadata rather than blocking segment dispatch. Physiology supports analytics without becoming a hidden dependency of reward computation.

DRIFT POLLING SPECIFICATION

PARAMETER	VALUE
Polling interval	30 seconds (DRIFT_POLL_INTERVAL)
ADB command	adb shell 'echo $EPOCHREALTIME'
Return format	Decimal epoch string with microsecond precision parsed as float
Maximum tolerated drift	150 milliseconds (MAX_TOLERATED_DRIFT_MS)
Drift formula	drift_offset = host_utc - android_epoch
Timestamp correction	corrected_ts = original_ts + drift_offset
Freeze threshold	3 consecutive ADB failures (DRIFT_FREEZE_AFTER_FAILURES)
Reset timeout	300 seconds / 5 minutes of frozen state (DRIFT_RESET_TIMEOUT)

FFMPEG RESAMPLING COMMAND

FFmpeg resampling command — BASH

```bash
ffmpeg -f s16le -ar 48000 -ac 1 -i pipe:0 -ar 16000 -f s16le -ac 1 pipe:1
```

CONTRACT REFERENCE

BOUNDARY	CONTRACT REFERENCE
INPUTS	Transient raw PCM capture stream, local H.264 video frames decoded inside module_c_orchestrator, validated event DTOs, Android epoch timestamps from drift polling, and local experiment bundle/cache state for arm selection.
OUTPUTS	DesktopIPCControlMessage queue records, SharedMemory 30-second PCM windows, deterministic segment_id, stable segment window bounds, AU12 series, stimulus-response metadata, _bandit_decision_snapshot, and optional _physiological_context.
OWNER	module_c_orchestrator
RUNTIME BOUNDARY	v4.0 desktop process graph

Failure behavior is summarized in this module's narrative and in the error-handling matrix; detailed failure-mode items remain in the embedded machine-readable artifact.

§4.D Multimodal delivery validation and response inference

Module D is the bounded ML interpretation layer. The `gpu_ml_worker` process consumes shared-memory PCM and schema-validated handoff metadata to produce transcription, acoustic observations,

delivery validation, and response inference. Heavy ML dependencies live here, while SQLite writes and persistent free-form rationale text remain outside the process boundary.

Purpose. Consume desktop IPC handoff messages and SharedMemory PCM windows to perform speech transcription, observational acoustic analysis, delivery validation, and bounded response inference.

Technology stack. faster-whisper 1.2.1, CTranslate2 >=4.5.0, PyTorch 2.4.x, sentence-transformers, transformers, praat-parselmouth, CUDA 12.x/cuDNN 9

§4.D.1 SPEECH TRANSCRIPTION

Speech transcription consumes the shared-memory PCM window and emits bounded transcript evidence for semantic validation and response observation. The transcript is derived text attached to a segment; the raw audio window remains transient and is zeroized after ownership closes. Compute-type handling and heavy dependency placement stay inside `gpu_ml_worker`.

§4.D.2 LEGACY COMPUTER VISION COMPATIBILITY NOTE

Historical computer-vision language is retained only for compatibility with older spec text. In v4, visual processing used by the live path is limited to derived telemetry such as AU12 observations and validity flags. Desktop visual analytics do not revive legacy frame persistence or container-era assumptions.

§4.D.3 ACOUSTIC ANALYSIS

Acoustic analysis is observational and validity-gated. Module D computes pitch, perturbation, voiced coverage, and windowed deltas only when the audio span is sufficient; otherwise it emits explicit false/null outputs. These values are analytics and attribution evidence, not reward-gating inputs in v4.0.

§4.D.4 DELIVERY VALIDATION AND RESPONSE INFERENCE PRE-PROCESSING

Pre-processing prepares the semantic boundary that keeps delivery validation and subject response separate. The top-level delivery-validation fields answer whether the selected stimulus was delivered for reward purposes, while `response_inference` records observed subject response within the configured horizon. Bounded reason codes and `pending`/terminal response states do not mutate §7B reward updates.

CONTRACT REFERENCE

BOUNDARY	CONTRACT REFERENCE
INPUTS	DesktopIPCControlMessage records containing segment_id, shared_memory_name, byte_length, checksum, AU12 telemetry, stimulus timing, expected_greeting compatibility field, generalized stimulus/response metadata, and bandit snapshot.
OUTPUTS	MLInferenceResult DTOs with transcript, bounded top-level delivery-validation fields reasoning/is_match/confidence_score, observational response_inference, acoustic analytics, and SharedMemory consumption acknowledgements.
OWNER	gpu_ml_worker
RUNTIME BOUNDARY	v4.0 desktop process graph

Failure behavior is summarized in this module's narrative and in the error-handling matrix; detailed failure-mode items remain in the embedded machine-readable artifact.

§4.E Experimentation & analytics

Module E is the local analytics and experiment-state owner. The `analytics_state_worker` process computes §7B rewards, updates Thompson Sampling posteriors, persists attribution and observational analytics, and prepares idempotent posterior deltas for cloud upload. Single-writer authority keeps replay, locking, and privacy filtering understandable.

Purpose. Aggregate ML outputs into local derived analytics, compute §7B delivery-gated rewards, update Thompson Sampling state, persist attribution analytics, and prepare cloud outbox records.

Technology stack. sqlite3, Pydantic schemas, pandas, SciPy, numpy

§4.E.1 EXECUTION DETAILS

Module E serializes derived analytics into local SQLite state through one writer process. Reward computation, posterior updates, attribution records, response-inference analytics, and `pending` upload rows become one ordered state transition stream. New persisted values must be classifiable as reward-path, attribution-path, dashboard-only, or cloud-sync material.

§4.E.2 PHYSIOLOGICAL CO-MODULATION ANALYTICS

Co-modulation uses only valid, non-stale derived physiological snapshots. Insufficient paired samples, stale context, or zero variance becomes a null-valid analytic outcome rather than an exception. Dashboards and attribution logic depend on preserving the difference between a missing signal and a failed subsystem.

§4.E.3 ATTRIBUTION ANALYTICS

Attribution analytics connect stimulus-linked events to later outcomes without changing online reward semantics. Module E persists AttributionEvent, OutcomeEvent, EventOutcomeLink, and AttributionScore

records with bounded method metadata and finality. Provisional and final replay paths stay idempotent, and observational attribution scores do not imply causal reward updates.

CONTRACT REFERENCE

BOUNDARY	CONTRACT REFERENCE
INPUTS	Validated MLInferenceResult and InferenceHandoffPayload-derived records, top-level delivery-validation fields, observational response_inference, physiological summaries, delayed outcome records, and operator state commands.
OUTPUTS	SQLite-derived analytics tables, RewardResult records, Bandit posterior updates, AttributionEvent / OutcomeEvent / EventOutcomeLink / AttributionScore records, posterior_delta records, pending_uploads rows, and operator read-model notifications.
OWNER	analytics_state_worker
RUNTIME BOUNDARY	v4.0 desktop process graph

Failure behavior is summarized in this module's narrative and in the error-handling matrix; detailed failure-mode items remain in the embedded machine-readable artifact.

§4.F Cloud synchronization

Module F is the offline-tolerant cloud synchronization layer. The `cloud_sync_worker` process drains derived telemetry, posterior deltas, and session lifecycle messages from the local outbox while refreshing signed experiment bundles. Network or cloud failure does not stop local inference and analytics, which continue from SQLite state and cached verified policy.

Purpose. Synchronize local derived telemetry, sessions, posterior deltas, OAuth state, and signed experiment bundles between the desktop outbox/cache and the cloud control plane.

Technology stack. httpx, keyring, cryptography, Pydantic schemas, SQLite outbox read models

§4.F.1 CLOUD SYNCHRONIZATION DETAILS

Cloud synchronization is a drain-and-refresh loop around local state, not a live control plane for segment processing. Module F uploads bounded telemetry and posterior deltas, fetches signed experiment bundles, refreshes OAuth tokens, and records acknowledgements through the local state boundary. Retries are idempotent, privacy rejection is explicit, and offline mode leaves the desktop runtime useful.

CONTRACT REFERENCE

BOUNDARY	CONTRACT REFERENCE
INPUTS	Pending upload notifications, auth commands, local pending_uploads rows, posterior_delta records, session lifecycle events, and signed ExperimentBundle responses from the cloud control plane.
OUTPUTS	Sync status, cloud health events, upload acknowledgements, cloud error records, refreshed tokens in Windows Credential Manager, and verified ExperimentBundle updates for analytics_state_worker to apply.
OWNER	cloud_sync_worker
RUNTIME BOUNDARY	v4.0 desktop process graph

Failure behavior is summarized in this module's narrative and in the error-handling matrix; detailed failure-mode items remain in the embedded machine-readable artifact.

SECTION 5

Data governance and security

Data governance protects the boundary between transient sensing and durable analytics. Raw biometric media, raw provider bodies, complete physiological chunks, and free-form rationales remain outside local and cloud persistence. SQLite stores bounded derived state for replay and operator readback, while cloud synchronization receives only privacy-filtered records that survive schema validation.

LSIE-MLF v4.0 processes sensitive speech, facial-affect, physiological, and behavioral data on a Windows desktop host. The system enforces local data minimization, volatile-media handling, SQLite derived-state limits, cloud schema-on-write rejection, encryption, and retention controls.

§5.4 Desktop local state

SQLite WAL is the durable local state boundary for the desktop runtime. The `analytics_state_worker` process owns writes, other processes use read-only access where allowed, and forbidden content rules prevent raw biometric media or unbounded rationale text from entering local persistence.

BOUNDARY	SPECIFICATION
Database	SQLite WAL
Single writer	analytics_state_worker
Read-only processes	ui_api_shell, cloud_sync_worker

BOUNDARY	SPECIFICATION
Forbidden content	['raw PCM', 'decoded H.264 frames', 'raw MKV or H.264 byte streams', 'voiceprints', 'raw Oura webhook JSON', 'hydrated provider response bodies', 'complete PhysiologicalChunkEvent payloads', 'free-form semantic rationales', 'free-form response rationales', 'unbounded stimulus payload content', 'OAuth refresh tokens']

§5.1 Ephemeral vault specification

PARAMETER	SPECIFICATION
Raw media residency	Process memory, FFmpeg pipes, or SharedMemory only
SharedMemory lifecycle	Owner zeroizes, closes, and unlinks after acknowledgement
SQLite mode	WAL with single writer
Cloud perimeter	Derived telemetry only
Secrets	Windows Credential Manager

§5.2 Data classification

Transient media

PCM windows, H.264 MKV stream bytes, decoded frames

STORAGE RULE Allowed only in process memory, FFmpeg pipes, or SharedMemory; never SQLite, cloud storage, logs, or crash dumps.

Local derived state

metrics, posterior state, attribution, physiology summaries, bounded response_inference, pending uploads

STORAGE RULE SQLite WAL, written only by analytics_state_worker. Free-form semantic rationales, free-form response rationales, and unbounded stimulus payload content are forbidden.

Cloud derived telemetry

segment telemetry, posterior deltas, experiment outcomes, bounded stimulus-response analytics

STORAGE RULE Cloud PostgreSQL only through HTTPS ingest APIs with schema-on-write rejection of local forbidden content.

Secrets

OAuth refresh token, cloud client credentials

STORAGE RULE Windows Credential Manager via keyring only; never SQLite, logs, pending_uploads, or exported operator files.

Forbidden persistent content

raw PCM, decoded H.264 frames, raw MKV, raw Oura JSON, complete PhysiologicalChunkEvent, free-form semantic rationales, free-form response rationales, unbounded stimulus payload content

STORAGE RULE must be rejected locally and by cloud API models.

The system must not store raw facial images, decoded video frames, raw H.264/MKV media, raw PCM, voiceprints, raw Oura webhook JSON, hydrated provider response bodies, complete transient PhysiologicalChunkEvent payload bodies, free-form semantic rationales, free-form response rationales, or unbounded stimulus payload content in SQLite, cloud PostgreSQL, logs, crash dumps, `pending` upload queues, or operator exports. Persistent storage is limited to derived, normalized outputs such as AU12 metrics, scalar RMSSD, heart rate, bounded semantic reason codes, semantic probability estimates, co-modulation metrics, acoustic metrics, posterior deltas, and attribution analytics.

SECTION 6

Interface contracts and payload schemas

Interface contracts are the typed language spoken between desktop processes, ML workers, analytics state, and cloud services. The rendered registry identifies which boundary each contract governs, while the embedded JSON Schema remains authoritative for field names, required keys, and validation behavior. Typed payloads are the rule for handoffs, control messages, telemetry records, posterior deltas, and attribution events.

Distributed multimodal pipelines require strict interface contracts. v4.0 keeps InferenceHandoffPayload as the canonical segment contract while changing active desktop audio transport to Queue-plus-SharedMemory IPC.

The active v4.0 desktop runtime uses DesktopIPCControlMessage for local process transport. Audio bytes must live in SharedMemory referenced by `shared_memory_name`, `byte_length`, and `checksum`; `_audio_data` must not appear in queue messages.

Top-level `reasoning`, `is_match`, and `confidence_score` remain the delivery-validation contract and the §7B reward gate. The `response_inference` object is bounded observational data for subject-response analytics and must not modulate §7B rewards.

For replay compatibility, `_expected_greeting` and expected_greeting remain required legacy compatibility fields. New generalized records should derive those compatibility values from the canonical expected stimulus rule when an active arm defines one.

DesktopIPCControlMessage is defined once in this section and reused verbatim by runtime_topology.ipc_contract.control_message_schema; producers and consumers must not maintain divergent interface and runtime variants.

§6.1–6.4 Schema registry

The human PDF renders a contract registry rather than full JSON Schema sections. Detailed validation anchors remain in the embedded machine-readable artifact and are not displayed as reader-facing section references when the underlying schema bodies are omitted.

CONTRACT	RENDERED BOUNDARY	HUMAN-READABLE DETAIL
InferenceHandoffPayload	Segment handoff from `module_c_orchestrator` into ML interpretation and analytics state.	Identifies the segment, time window, stimulus attempt, delivery validation, optional physiology, bounded semantic outputs, and observational response evidence carried downstream.
BanditDecisionSnapshot	Pre-update Thompson Sampling decision snapshot attached to the segment payload.	Freezes arm identity, posterior state, and selection context before outcomes are observed so replay and attribution can explain the decision deterministically.
PhysiologicalChunkEvent	Wearable telemetry ingress record consumed by the physiology buffer before segment assembly.	Normalizes provider chunks into bounded, role-scoped records that can produce scalar freshness-aware physiological context without persisting raw biometric media.
PhysiologicalContext	Optional per-segment scalar physiology snapshot carried on eligible inference handoffs.	Carries RMSSD, heart-rate, validity, freshness, provider, and derivation metadata while allowing stale or absent physiology to remain non-blocking.
AttributionEvent	Stimulus-linked interaction record entering the attribution ledger.	Captures the event identity, stimulus context, semantic method metadata, bounded reason code, and eligibility fields used for later outcome association.
OutcomeEvent	Delayed platform outcome record that may be linked back to an earlier attribution event.	Represents outcomes such as follows with stable identity and timestamps, kept separate from immediate stimulus-response observations.
EventOutcomeLink	Deterministic eligibility link between an attribution event and an outcome event.	Records the versioned rule, horizon, and finality state that explain why a downstream outcome is eligible for a given stimulus-linked event.
AttributionScore	Method-specific observational attribution score emitted after event/outcome association.	Stores bounded semantic or observational scores for analysis without feeding them back into the online §7B reward update.

CONTRACT	RENDERED BOUNDARY	HUMAN-READABLE DETAIL
DesktopIPCControlMessage	Queue-level control envelope for local IPC queues/shared-memory blocks in the desktop ProcessGraph.	References shared-memory audio by name, byte length, and checksum while keeping raw audio bytes out of queue messages.
ExperimentBundle	Signed cloud-delivered experiment configuration cached by the desktop runtime.	Defines the arm set, bundle identity, signature context, and local cache boundary used when the desktop runtime refreshes experiment configuration.
PosteriorDelta	Idempotent cloud outbox upload for locally computed posterior changes.	Carries posterior updates from SQLite local state to cloud sync without making the cloud service part of the local reward-update critical path.

SECTION 7

Mathematical specifications

§7A AU12 facial action unit geometry

Facial geometry yields reproducible AU12 intensity from landmark positions. Smile-corner motion is normalized against interocular distance so downstream reward and attribution records use a stable, scale-aware signal.

VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
L_i	`landmarks[i]`	3D vector (x_i, y_i, z_i) of MediaPipe landmark i.
L_{61}	`landmarks[61]`	Left lip corner landmark.
L_{291}	`landmarks[291]`	Right lip corner landmark.
L_{33}	`landmarks[33]`	Left eye outer corner.
L_{133}	`landmarks[133]`	Left eye inner corner.
L_{362}	`landmarks[362]`	Right eye inner corner.
L_{263}	`landmarks[263]`	Right eye outer corner.
E_L	`e_l`	Geometric center of left eye midpoint(L_33,L_133).
E_R	`e_r`	Geometric center of right eye midpoint(L_362,L_263).
IOD	`iod`	3D Euclidean distance between E_L and E_R.
D_{mouth}	`d_mouth`	3D Euclidean distance between lip corners.
B_{neutral}	`self.b_neutral`	Neutral baseline average of D_mouth/IOD.
$I_{\text{AU12}}(t)$	`i_au12`	Normalized AU12 intensity at time t.

SYMBOL	CODE NAME	DEFINITION
α	`self.alpha`	Empirical linear projection multiplier. Default 6.0; calibrated so a Duchenne deviation of approximately 0.15 maps to approximately 0.90 before tanh bounding.
$Score_{\text{AU12}}(t)$	`score_au12`	Final AU12 FACS score in range 0–5.
$Score_{\text{AU12,bounded}}(t)$	`compute_bounded_intensity()`	AU12 intensity bounded to [0.0, 1.0] via tanh soft-saturation. Output method for the fractional Beta-Bernoulli Thompson Sampling reward pipeline.

STEP 1: TOPOLOGICAL LANDMARK EXTRACTION

The anatomical points required for AU12 computation are extracted from the MediaPipe landmark array.

STEP 2: DERIVATION OF INTEROCULAR DISTANCE (IOD)

Eye centers are computed as midpoints between inner and outer eye landmarks and used to calculate the normalization scalar, with the accompanying formula giving the normative computation.

$$E_L = ((x_{33}+x_{133})/2, (y_{33}+y_{133})/2, (z_{33}+z_{133})/2); \quad E_R = ((x_{362}+x_{263})/2, (y_{362}+y_{263})/2, (z_{362}+z_{263})/2)$$

$$IOD = \sqrt{(E_R.x - E_L.x)^2 + (E_R.y - E_L.y)^2 + (E_R.z - E_L.z)^2}$$

STEP 3: AU12 DISTANCE AND BASELINE CALIBRATION

The raw AU12 distance equals the Euclidean distance between lip corners, with the accompanying formula giving the normative computation.

$$D_{\text{mouth}} = \sqrt{(x_{291} - x_{61})^2 + (y_{291} - y_{61})^2 + (z_{291} - z_{61})^2}$$

$$I_{\text{AU12}}(t) = (D_{\text{mouth}}(t)/IOD(t)) - B_{\text{neutral}}$$

$$Score_{\text{AU12}}(t) = \min\left(5, \max(0, \alpha \cdot I_{\text{AU12}}(t))\right)$$

$$Score_{\text{AU12,bounded}}(t) = \tanh\big(\alpha \cdot \max(0,\ I_{\text{AU12}})\big)$$

AU12NORMALIZER — REFERENCE IMPLEMENTATION

Reference implementation PYTHON

```python
from __future__ import annotations
import math
from typing import Any
import numpy as np
import numpy.typing as npt

EPSILON: float = 1e-6
DEFAULT_ALPHA_SCALE: float = 6.0  # §7A: 0.90 / 0.15 = 6.0


class AU12Normalizer:
    def __init__(self, alpha: float = DEFAULT_ALPHA_SCALE) -> None:
        self.alpha: float = alpha
        self.b_neutral: float | None = None
        self.calibration_buffer: list[float] = []

    def _extract_geometry(
        self, landmarks: npt.NDArray[np.floating[Any]]
    ) -> tuple[float, float] | None:
        left_eye_outer = landmarks[33]
        left_eye_inner = landmarks[133]
        right_eye_inner = landmarks[362]
        right_eye_outer = landmarks[263]
        left_lip = landmarks[61]
        right_lip = landmarks[291]

        e_l = (left_eye_outer + left_eye_inner) / 2.0
        e_r = (right_eye_inner + right_eye_outer) / 2.0
        iod: float = float(np.linalg.norm(e_r - e_l))
        if iod < EPSILON:
            return None
        d_mouth: float = float(np.linalg.norm(right_lip - left_lip))
        return (iod, d_mouth)

    def _update_calibration(self, ratio: float) -> None:
        self.calibration_buffer.append(ratio)
        self.b_neutral = float(np.mean(self.calibration_buffer))

    def compute_raw_ratio(
        self, landmarks: npt.NDArray[np.floating[Any]]
    ) -> float | None:
        geom = self._extract_geometry(landmarks)
        if geom is None:
            return None
        iod, d_mouth = geom
        return d_mouth / iod

    def compute_intensity(
        self, landmarks: npt.NDArray[np.floating[Any]], is_calibrating: bool = False
    ) -> float:
        geom = self._extract_geometry(landmarks)
        if geom is None:
            return 0.0
        iod, d_mouth = geom
        ratio = d_mouth / iod
        if is_calibrating:
            self._update_calibration(ratio)
            return 0.0
```

```
        if self.b_neutral is None:
            raise ValueError("Baseline not calibrated")
        score = self.alpha * (ratio - self.b_neutral)
        return float(min(max(score, 0.0), 5.0))

    def compute_bounded_intensity(
        self, landmarks: npt.NDArray[np.floating[Any]], is_calibrating: bool = False
    ) -> float:
        geom = self._extract_geometry(landmarks)
        if geom is None:
            return 0.0
        iod, d_mouth = geom
        ratio = d_mouth / iod
        if is_calibrating:
            self._update_calibration(ratio)
            return 0.0
        if self.b_neutral is None:
            raise ValueError("Baseline not calibrated")
        deviation = max(0.0, ratio - self.b_neutral)
        return float(math.tanh(self.alpha * deviation))
```

§7B Thompson sampling reward pipeline

Active stimulus selection uses delivery validation and AU12 intensity to compute the gated reward and posterior update. Response inference remains observational and does not change the Thompson Sampling posterior.

VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
t_{stim}	`stimulus_time_s`	Drift-corrected UTC epoch timestamp of the selected stimulus trigger. The compatibility field _stimulus_time is formally interpreted as trigger_time.
W_{measure}	`extract_stimulus_window()`	Stimulus-locked measurement window [*t_stim* + 0.5s, *t_stim* + 5.0s].
W_{baseline}	`extract_stimulus_-window(baseline)`	Pre-stimulus baseline window [*t_stim* - 5.0s, *t_stim* - 2.0s].
P_{90}	`p90_intensity`	90th percentile of bounded AU12 intensities within *W_measure*.
G_t	`semantic_gate`	Legacy code name for the binary delivery-validation gate: 1 if top-level `is_match` is True, 0 otherwise.
r_t	`gated_reward`	Normative §7B reward signal: *r_t* = *P_90* × *G_t*. Range [0.0, 1.0]. Response-inference outputs do not alter this value.
α	`posterior_alpha`	Beta distribution alpha parameter (success count). Prior: 1.0.
β	`posterior_beta`	Beta distribution beta parameter (failure count). Prior: 1.0.

SYMBOL	CODE NAME	DEFINITION
$B_{\text{neutral,pre}}$	`au12_baseline_pre`	Mean bounded AU12 intensity in *W_baseline*, or null when no pre-stimulus frames exist.
$seed_t$	`random_seed`	Unsigned 64-bit deterministic seed derived as int.from_bytes(SHA256(session_id || segment_window_start_utc || stimulus_time || policy_version).digest()[:8], 'big').
R^{obs}	`response_inference.is_match`	Observational boolean indicating whether qualifying subject-response evidence was inferred under the expected response rule.
$t^{matched_{\text{response}}}$	`response_inference.matched_response_time`	Timestamp of the matched observable subject response, or null when no qualifying response is identified.
Δt^{resp}	`response_latency_s`	Observed response latency in seconds relative to trigger_time, or null when no matched response exists.
A^{resp}	`response_locked_affect_score`	Observational response-locked AU12 P90 score computed after matched_response_time when present.
$r^{resp_{\text{candidate}}}$	`combined_response_reward_candidate`	Observational candidate combining response inference and response-locked affect; not used for §7B posterior updates in v4.0.

STEP 1: SIGNAL SOURCE AND BOUNDED OUTPUT

The reward pipeline consumes per-frame bounded AU12 intensities from `compute_bounded_intensity()` (§7A). Each observation is a `{timestamp_s, intensity}` pair with intensity in [0.0, 1.0].

STEP 2: STIMULUS WINDOW EXTRACTION

The measurement window is defined as [*t_stim* + 0.5 seconds, *t_stim* + 5.0 seconds]. The 0.5-second onset delay accounts for physiological latency after the validated stimulus trigger. The 5.0-second end bound captures the full smile trajectory including onset, apex, and initial decay, with the accompanying formula giving the normative computation.

$$W_{\text{measure}} = \{(t_i, s_i) \in \text{au12_series} \mid t_{\text{stim}} + 0.5 \leq t_i \leq t_{\text{stim}} + 5.0\}$$

$$W_{\text{baseline}} = \{(t_i, s_i) \in \text{au12_series} \mid t_{\text{stim}} - 5.0 \leq t_i \leq t_{\text{stim}} - 2.0\}$$

STEP 3: P90 AGGREGATION AND GATED REWARD

The 90th percentile is used instead of the mean or maximum to reduce sensitivity to single-frame noise spikes while still capturing the peak smile response. If the measurement window contains zero frames, P90 defaults to 0.0, with the accompanying formula giving the normative computation.

$$P_{90} = \text{percentile}_{90}\big(\{s_i \mid (t_i, s_i) \in W_{\text{measure}}\}\big)$$

$$G_t = \begin{cases} 1 & \text{if is_match} = \text{True} \\ 0 & \text{otherwise} \end{cases}$$

$$r_t = P_{90} \times G_t$$

STEP 4: FRACTIONAL BETA-BERNOULLI POSTERIOR UPDATE

Each arm maintains a Beta posterior initialized with a uniform prior Beta(1, 1). After each encounter, the posterior is updated with the fractional reward *r_t* ∈ [0, 1]. Unlike standard Bernoulli updates that require binary 0/1 outcomes, the fractional update preserves the continuous gradient of the bounded AU12 signal, with the accompanying formula giving the normative computation.

$$\alpha \leftarrow \alpha + r_t, \quad \beta \leftarrow \beta + (1 - r_t)$$

$$a^* = \arg\max_a \theta_a, \quad \theta_a \sim \text{Beta}(\alpha_a, \beta_a)$$

STEP 5: DETERMINISTIC OFFLINE ARM SELECTION SEED

When the cloud control plane is unavailable, offline arm selection remains deterministic and replayable. The `module_c_orchestrator` process derives `random_seed` before selection and stores it in BanditDecisionSnapshot, with the accompanying policy version and candidate-arm set needed to reconstruct the selection.

$$seed_t = \text{int64}_{\text{be}}(\text{SHA256}(session_id \| segment_window_start_utc \| stimulus_time \| policy_version)[0:8])$$

STEP 6: OBSERVATIONAL RESPONSE ANALYTICS

Response inference measures whether observable evidence indicates that the subject read, answered, acknowledged, or otherwise responded to the delivered stimulus under `expected_response_rule`, with the accompanying formula giving the normative computation.

$$R_i^{\text{obs}} = \begin{cases} 1 & \text{if response_inference}_i\text{.is_match} = \text{True} \\ 0 & \text{otherwise} \end{cases}$$

$$\Delta t_i^{\text{resp}} = t_i^{\text{matched_response}} - t_i^{\text{trigger}} \quad \text{when } t_i^{\text{matched_response}} \neq null$$

$$A_i^{\text{resp}} = \text{percentile}_{90}(\{s_j \mid t_i^{\text{matched_response}} + 0.5 \leq t_j \leq t_i^{\text{matched_response}} + 5.0\})$$

$$r_i^{\text{resp_candidate}} = R_i^{\text{obs}} \times A_i^{\text{resp}}$$

REWARD PIPELINE AND THOMPSON SAMPLING — REFERENCE IMPLEMENTATION

Reference implementation **PYTHON**

```python
from __future__ import annotations
import random
from dataclasses import dataclass
from typing import Optional
import numpy as np

STIMULUS_WINDOW_START: float = 0.5   # seconds after stimulus
STIMULUS_WINDOW_END: float = 5.0     # seconds after stimulus
BASELINE_WINDOW_START: float = -5.0  # seconds before stimulus
BASELINE_WINDOW_END: float = -2.0    # seconds before stimulus
DEFAULT_PRIOR_ALPHA: float = 1.0
DEFAULT_PRIOR_BETA: float = 1.0


@dataclass
class RewardResult:
    gated_reward: float
    p90_intensity: float
    semantic_gate: int
    n_frames_in_window: int
    au12_baseline_pre: Optional[float]


def extract_stimulus_window(
    au12_series: list[dict], stimulus_time_s: float,
    start_offset: float = STIMULUS_WINDOW_START,
    end_offset: float = STIMULUS_WINDOW_END,
) -> list[dict]:
    t_start = stimulus_time_s + start_offset
    t_end = stimulus_time_s + end_offset
    return [o for o in au12_series if t_start ≤ o["timestamp_s"] ≤ t_end]


def compute_reward(
    au12_series: list[dict], stimulus_time_s: float,
    is_match: bool,
) -> RewardResult:
    window = extract_stimulus_window(au12_series, stimulus_time_s)
    n_frames = len(window)

    if n_frames == 0:
        p90 = 0.0
    else:
        p90 = float(np.percentile([o["intensity"] for o in window], 90))


    gate = 1 if is_match else 0
    gated = p90 * gate

    baseline_obs = extract_stimulus_window(
        au12_series, stimulus_time_s,
        start_offset=BASELINE_WINDOW_START, end_offset=BASELINE_WINDOW_END,
    )
```

```
    baseline = (
        float(np.mean([o["intensity"] for o in baseline_obs]))
        if baseline_obs else None
    )

    return RewardResult(
        gated_reward=gated, p90_intensity=p90, semantic_gate=gate,
        n_frames_in_window=n_frames, au12_baseline_pre=baseline,
    )


def thompson_sample_arm(arms: dict[str, tuple[float, float]]) -> str:
    samples = {
        arm_id: random.betavariate(alpha, beta)
        for arm_id, (alpha, beta) in arms.items()
    }
    return max(samples, key=samples.get)


def update_posterior(
    alpha: float, beta: float, reward: float
) -> tuple[float, float]:
    return alpha + reward, beta + (1.0 - reward)
```

§7C Physiological co-modulation analytics

Streamer and operator physiological signals are aligned and correlated only when enough fresh, valid paired RMSSD samples exist. The Co-Modulation Index is a null-valid analytic outcome when coverage or variance is insufficient.

VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
$R_s^{(k)}$	`streamer_rmssd_bins[k]`	Streamer RMSSD in a 1-minute resampled analysis bin.
$R_o^{(k)}$	`operator_rmssd_bins[k]`	Operator RMSSD in a 1-minute resampled analysis bin.
C_{mod}	`co_modulation_index`	Pearson correlation between aligned streamer and operator RMSSD sequences.
N	`n_paired_observations`	Number of aligned paired observations in the analysis window.
$coverage$_ratio	`coverage_ratio`	Fraction of the analysis window with valid paired observations.
MIN_COMOD_PAIRS	`MIN_COMOD_PAIRS`	Minimum aligned pairs required for a valid correlation.
$COMOD$_WINDOW_MINUTES	`COMOD_WINDOW_MINUTES`	Rolling analysis window duration in minutes.

STEP 1: DATA SOURCE AND ELIGIBILITY

Use per-segment physiological snapshots from `physiology_log`.

STEP 2: WINDOWING, RESAMPLING, AND TIME ALIGNMENT

For each subject, select eligible snapshots in the 10-minute window ending at `window_end_utc`, with the accompanying formula giving the normative computation.

$$R_s = \text{resample}_{1\text{min}}(\text{streamer_rmssd})$$

$$R_o = \text{resample}_{1\text{min}}(\text{operator_rmssd})$$

STEP 3: PEARSON CORRELATION AND NULL GATING

Compute Pearson correlation on the aligned RMSSD series. Positive values indicate alignment; negative values indicate divergence, with the accompanying formula giving the normative computation.

$$C_{\text{mod}} = \frac{\sum(R_s^{(k)} - \bar{R}_s)(R_o^{(k)} - \bar{R}_o)}{\sqrt{\sum(R_s^{(k)} - \bar{R}_s)^2 \sum(R_o^{(k)} - \bar{R}_o)^2}}$$

STEP 4: COVERAGE RATIO AND PERSISTENCE

Coverage ratio measures the fraction of the window with valid paired observations, with the accompanying formula giving the normative computation.

$$\text{coverage_ratio} = \frac{N}{\max(|R_s|, |R_o|, 1)}$$

STEP 5: SCOPE CONSTRAINT

This analytics path is observational only under the §7B reward-path invariance rule.

PHYSIOLOGICAL CO-MODULATION — REFERENCE IMPLEMENTATION

Reference implementation PYTHON

```python
from __future__ import annotations

from dataclasses import dataclass
from typing import Optional
```

```python
import pandas as pd
from scipy.stats import pearsonr

MIN_COMOD_PAIRS: int = 4
COMOD_WINDOW_MINUTES: int = 10
RESAMPLE_RULE: str = "1min"


@dataclass(frozen=True)
class CoModulationResult:
    co_modulation_index: Optional[float]
    n_paired_observations: int
    coverage_ratio: float
    streamer_rmssd_mean: Optional[float]
    operator_rmssd_mean: Optional[float]


def _resample_rmssd(window_df: pd.DataFrame, subject_role: str) -> pd.Series:
    role_df = (
        window_df.loc[
            window_df["subject_role"] == subject_role,
            ["source_timestamp_utc", "rmssd_ms"],
        ]
        .dropna(subset=["rmssd_ms"])
        .sort_values("source_timestamp_utc")
    )

    if role_df.empty:
        return pd.Series(dtype="float64")

    return (
        role_df.set_index("source_timestamp_utc")["rmssd_ms"]
        .resample(RESAMPLE_RULE)
        .mean()
        .dropna()
    )


def compute_comodulation(
    physiology_log: pd.DataFrame,
    window_end_utc: pd.Timestamp,
) -> CoModulationResult:
    window_end_utc = pd.Timestamp(window_end_utc, tz="UTC")
    window_start_utc = window_end_utc - pd.Timedelta(minutes=COMOD_WINDOW_MINUTES)

    eligible = physiology_log.copy()
    eligible["source_timestamp_utc"] = pd.to_datetime(
        eligible["source_timestamp_utc"], utc=True
    )
    eligible = eligible.loc[
        (eligible["source_timestamp_utc"] > window_start_utc)
        & (eligible["source_timestamp_utc"] <= window_end_utc)
        & eligible["rmssd_ms"].notna()
        & (eligible["is_valid"] == True)
        & (eligible["is_stale"] == False)
    ]

    streamer = _resample_rmssd(eligible, "streamer")
    operator = _resample_rmssd(eligible, "operator")
```

```
    aligned = pd.concat(
        [streamer.rename("streamer"), operator.rename("operator")],
        axis=1,
        join="inner",
    ).dropna()

    n_pairs = int(len(aligned))
    coverage_ratio = float(
        n_pairs / max(len(streamer), len(operator), 1)
    )

    streamer_mean = float(streamer.mean()) if not streamer.empty else None
    operator_mean = float(operator.mean()) if not operator.empty else None

    if n_pairs < MIN_COMOD_PAIRS:
        return CoModulationResult(
            co_modulation_index=None,
            n_paired_observations=n_pairs,
            coverage_ratio=coverage_ratio,
            streamer_rmssd_mean=streamer_mean,
            operator_rmssd_mean=operator_mean,
        )

    if aligned["streamer"].nunique() < 2 or aligned["operator"].nunique() < 2:
        return CoModulationResult(
            co_modulation_index=None,
            n_paired_observations=n_pairs,
            coverage_ratio=coverage_ratio,
            streamer_rmssd_mean=streamer_mean,
            operator_rmssd_mean=operator_mean,
        )

    co_modulation_index, _p_value = pearsonr(
        aligned["streamer"], aligned["operator"]
    )

    return CoModulationResult(
        co_modulation_index=float(co_modulation_index),
        n_paired_observations=n_pairs,
        coverage_ratio=coverage_ratio,
        streamer_rmssd_mean=streamer_mean,
        operator_rmssd_mean=operator_mean,
    )
```

§7D Observational acoustic analytics

Observational acoustic measurements are computed around stimulus windows. Voiced-coverage and perturbation validity gates prevent unstable audio spans from producing misleading pitch, jitter, or shimmer deltas.

VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
τ_{voice}	`ACOUSTIC_VOICING_THRESHOLD`	Minimum Praat voicing threshold for a frame to be treated as voiced.
$T_{\text{voice,min}}$	`ACOUSTIC_MIN_VOICED_COVERAGE_S`	Minimum cumulative voiced coverage required for F0 validity inside a window.
$T_{\text{island,min}}$	`ACOUSTIC_MIN_STABLE_ISLAND_S`	Minimum duration of one continuous voiced island required for F0 validity inside a window.
$N_{\text{peaks,min}}$	`ACOUSTIC_MIN_PERIODIC_PEAKS`	Minimum periodic peaks required for perturbation validity inside a selected voiced island.
$V(W)$	`voiced_frames(window)`	Frames in the analysis window whose voicing strength meets or exceeds the voicing threshold.
$C_{\text{voice}}(W)$	`voiced_coverage_s(window)`	Sum of durations of voiced frames in the analysis window.
$L_{\text{voice}}(W)$	`longest_voiced_island_s(window)`	Duration of the longest contiguous voiced island in the analysis window.
$P(W)$	`perturbation_span(window)`	Selected contiguous voiced island used for jitter/shimmer computation; it must satisfy perturbation validity constraints.
Δ_{ST}	`f0_delta_semitones`	Semitone difference between measurement-window and baseline-window mean F0.

STEP 1: STIMULUS WINDOWS AND VOICED-ONLY MASKING

Use the same stimulus-locked windows already defined by §7B, with the accompanying formula giving the normative computation.

$$W_{\text{measure}} = [t_{\text{stim}} + 0.5\text{s},\ t_{\text{stim}} + 5.0\text{s}]$$

$$W_{\text{baseline}} = [t_{\text{stim}} - 5.0\text{s},\ t_{\text{stim}} - 2.0\text{s}]$$

$$V(W) = \{\text{frame}_i \in W \mid \text{voicing_strength}(\text{frame}_i) \geq \tau_{\text{voice}}\}$$

$$C_{\text{voice}}(W) = \sum_{\text{frame}_i \in V(W)} \text{duration}(\text{frame}_i)$$

$$L_{\text{voice}}(W) = \text{max_contiguous_duration}(V(W))$$

STEP 2: VALIDITY GATING AND PERTURBATION-SPAN SELECTION

For each window W, F0 validity requires minimum cumulative voiced coverage and a minimum stable voiced island.

STEP 3: SUMMARY STATISTICS AND DELTA FORMULAS

Voiced coverage outputs are the cumulative voiced coverage in the measure and baseline windows, with the accompanying formula giving the normative computation.

$$f0_mean_measure_hz = \text{mean}(F0(\text{frame}_i) \mid \text{frame}_i \in V(W_{\text{measure}}),\ F0(\text{frame}_i) > 0)$$

$$f0_mean_baseline_hz = \text{mean}(F0(\text{frame}_i) \mid \text{frame}_i \in V(W_{\text{baseline}}),\ F0(\text{frame}_i) > 0)$$

$$\Delta_{\text{ST}} = 12 \cdot \log_2 \left(\frac{f0_mean_measure_hz}{f0_mean_baseline_hz} \right)$$

$$\text{jitter_delta} = \text{jitter_mean_measure} - \text{jitter_mean_baseline}$$

$$\text{shimmer_delta} = \text{shimmer_mean_measure} - \text{shimmer_mean_baseline}$$

STEP 4: NULL STIMULUS BEHAVIOR

If `_stimulus_time` is null, Module D emits deterministic false/null observational acoustic outputs instead of attempting stimulus-window analysis.

STEP 5: SCOPE CONSTRAINT

This analytics path is observational only under the §7B reward-path invariance rule.

OBSERVATIONAL ACOUSTIC ANALYTICS — REFERENCE IMPLEMENTATION

Reference implementation PYTHON

```python
from __future__ import annotations

from dataclasses import dataclass
from math import log2
from typing import Optional

ACOUSTIC_VOICING_THRESHOLD: float = 0.45
ACOUSTIC_MIN_VOICED_COVERAGE_S: float = 1.0
ACOUSTIC_MIN_STABLE_ISLAND_S: float = 0.2
ACOUSTIC_MIN_PERIODIC_PEAKS: int = 4

STIMULUS_WINDOW_START: float = 0.5
STIMULUS_WINDOW_END: float = 5.0
BASELINE_WINDOW_START: float = -5.0
BASELINE_WINDOW_END: float = -2.0


@dataclass(frozen=True)
class AcousticAnalyticsResult:
    f0_valid_measure: bool
    f0_valid_baseline: bool
    perturbation_valid_measure: bool
    perturbation_valid_baseline: bool
    voiced_coverage_measure_s: float
    voiced_coverage_baseline_s: float
    f0_mean_measure_hz: Optional[float]
    f0_mean_baseline_hz: Optional[float]
    f0_delta_semitones: Optional[float]
```

```
    jitter_mean_measure: Optional[float]
    jitter_mean_baseline: Optional[float]
    jitter_delta: Optional[float]
    shimmer_mean_measure: Optional[float]
    shimmer_mean_baseline: Optional[float]
    shimmer_delta: Optional[float]


def compute_f0_delta_semitones(
    f0_mean_measure_hz: Optional[float],
    f0_mean_baseline_hz: Optional[float],
) -> Optional[float]:
    if f0_mean_measure_hz is None or f0_mean_baseline_hz is None:
        return None
    if f0_mean_baseline_hz ≤ 0:
        return None
    return 12.0 * log2(f0_mean_measure_hz / f0_mean_baseline_hz)


def null_acoustic_result() -> AcousticAnalyticsResult:
    return AcousticAnalyticsResult(
        f0_valid_measure=False,
        f0_valid_baseline=False,
        perturbation_valid_measure=False,
        perturbation_valid_baseline=False,
        voiced_coverage_measure_s=0.0,
        voiced_coverage_baseline_s=0.0,
        f0_mean_measure_hz=None,
        f0_mean_baseline_hz=None,
        f0_delta_semitones=None,
        jitter_mean_measure=None,
        jitter_mean_baseline=None,
        jitter_delta=None,
        shimmer_mean_measure=None,
        shimmer_mean_baseline=None,
        shimmer_delta=None,
    )
```

§7E Event→Outcome attribution policy

Stimulus-linked events can be connected to later outcomes without changing the online reward path. The attribution ledger separates delivery validation, AU12, response inference, synchrony, and finality so analysis can be replayed deterministically.

VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
$segment_{\text{id}}$	`segment_id`	Deterministic SHA-256 segment identifier derived from stable segment identity fields.
H_{attr}	`attribution_horizon_s`	Maximum eligible event→outcome attribution horizon in seconds.

SYMBOL	CODE NAME	DEFINITION
lag_s	`lag_s`	Time difference between outcome timestamp and event timestamp in seconds.
p_{match}	`confidence_score`	Delivery-validation probability estimate produced by the active deterministic method.
r_t^{soft}	`soft_reward_candidate`	Observational soft delivery-validation reward candidate persisted under §7E only.
B_{pre}	`au12_baseline_pre`	Mean bounded AU12 intensity in the pre-stimulus baseline window.
L_{p90}	`au12_lift_p90`	Baseline-aware lift using the post-stimulus P90 bounded AU12 intensity.
L_{peak}	`au12_lift_peak`	Baseline-aware lift using the post-stimulus peak bounded AU12 intensity.
$\Delta t_{\text{peak,ms}}$	`au12_peak_latency_ms`	Latency from stimulus timestamp to peak AU12 observation in milliseconds.
$\rho(\ell)$	`lagged_correlation`	Lagged Pearson correlation at lag \ell.
S	`sync_peak_corr`	Maximum lagged correlation over the configured lag search range.
$finality$	`finality`	Attribution record lifecycle state: online_provisional or offline_final.
r_t^{proxy}	`proxy_reward`	The observational proxy reward candidate used for attribution scoring, typically the §7B gated_reward or a documented diagnostic component. Response-inference candidates are observational only.
x_t	`source_series`	First time-aligned observational signal used in lag-aware synchrony analytics.
y_t	`target_series`	Second time-aligned observational signal used in lag-aware synchrony analytics.
$response_{\text{status}}$	`response_registration_status`	Bounded response-inference state describing whether qualifying observable response evidence was found.
$response_{\text{reason}}$	`response_reason_code`	Bounded reason code scoped to response inference; not a top-level delivery-validation reason.
t_{response}	`matched_response_time`	Timestamp of the matched observable subject response, or null when no qualifying response exists.
I_{response}	`observable_response_indicator`	Observational indicator derived from response_-inference.is_match for attribution analytics only.

STEP 1: STABLE EVENT IDENTITY AND SCOPE

Each dispatched interaction segment must carry a deterministic segment identity so reprocessing and offline linkage remain idempotent, with the accompanying formula giving the normative computation.

$$\text{segment_id} = \text{SHA256}(\text{session_id} \parallel \text{segment_window_start_utc} \parallel \text{segment_window_end_utc})$$

STEP 2: OUTCOME DEFINITION AND ELIGIBILITY LINKING

Delayed outcomes such as creator_follow are persisted as OutcomeEvent records and linked to prior events by deterministic eligibility rules, with the accompanying formula giving the normative computation.

$$\text{lag}_s = t_{\text{outcome}} - t_{\text{event}}$$

$$0 \leq \text{lag}_s \leq H_{\text{attr}}$$

STEP 3: DELIVERY-VALIDATION ATTRIBUTION SCORES

The delivery-validation scorer outputs a probability estimate *p_match* together with a bounded reason code and final top-level boolean gate, with the accompanying formula giving the normative computation.

$$r_t^{\text{soft}} = p_{\text{match}} \cdot r_t^{\text{proxy}}$$

$$P(\text{match} \mid \text{audio}, \text{rule}) = \sum_{k=1}^{K} P(\hat{u}_k \mid \text{audio})\, P_\phi(\text{match} \mid \hat{u}_k, \text{rule})$$

STEP 4: AU12 BASELINE-AWARE ATTRIBUTION METRICS

To distinguish stimulus-linked smiles from smiles already underway, attribution analytics persist baseline-aware AU12 lift metrics derived from the same trigger-locked windows already defined by §7B, with the accompanying formula giving the normative computation.

$$L_{\text{p90}} = \max(0, P_{90,post} - B_{\text{pre}})$$

$$L_{\text{peak}} = \max(0, P_{\text{max,post}} - B_{\text{pre}})$$

$$\Delta t_{\text{peak,ms}} = 1000 \cdot (t_{\text{peak}} - t_{\text{stim}})$$

STEP 5: RESPONSE-INFERENCE ATTRIBUTION METRICS

Response-inference attribution records whether observable evidence indicates that the subject responded to the delivered stimulus within the configured observation horizon, with the accompanying formula giving the normative computation.

$$\text{response_latency}_s = t_{\text{response}} - t_{\text{event}}$$

$$I_{\text{response}} = \mathbb{1}[\text{response_inference.is_match} = \text{True}]$$

STEP 6: LAG-AWARE SYNCHRONY ANALYTICS

Before more complex warping methods are introduced, attribution analytics persist a lagged-correlation baseline for observational synchrony, with the accompanying formula giving the normative computation.

$$\rho(\ell) = \mathrm{corr}(x_t, y_{t+\ell}), \qquad \ell \in [0, L]$$

$$S = \max_{\ell \in [0,L]} \rho(\ell)$$

STEP 7: FINALITY AND SCOPE CONSTRAINT

Attribution records transition from `online_provisional` to `offline_final` after attribution horizons close and finalization jobs complete.

ATTRIBUTION ANALYTICS — REFERENCE IMPLEMENTATION

Reference implementation PYTHON

```python
from __future__ import annotations

import hashlib
from dataclasses import dataclass
from typing import Optional

import numpy as np
from scipy.stats import pearsonr

STIMULUS_WINDOW_START: float = 0.5
STIMULUS_WINDOW_END: float = 5.0
BASELINE_WINDOW_START: float = -5.0
BASELINE_WINDOW_END: float = -2.0


def compute_segment_id(session_id: str, segment_window_start_utc: str, segment_window_end_utc:
↪  str) -> str:
    raw = f"{session_id}|{segment_window_start_utc}|{segment_window_end_utc}".encode("utf-8")
    return hashlib.sha256(raw).hexdigest()


def compute_soft_reward_candidate(p_match: Optional[float], proxy_reward: float) ->
↪  Optional[float]:
    if p_match is None:
        return None
    p = float(min(max(p_match, 0.0), 1.0))
    return p * float(proxy_reward)


def compute_au12_attribution_metrics(au12_series: list[dict], stimulus_time_s: float) ->
↪  dict[str, Optional[float]]:
```

```
    baseline_obs = [
        o["intensity"] for o in au12_series
        if stimulus_time_s + BASELINE_WINDOW_START ≤ o["timestamp_s"] ≤ stimulus_time_s +
        ↪  BASELINE_WINDOW_END
    ]
    measure_obs = [
        o for o in au12_series
        if stimulus_time_s + STIMULUS_WINDOW_START ≤ o["timestamp_s"] ≤ stimulus_time_s +
        ↪  STIMULUS_WINDOW_END
    ]
    if not measure_obs:
        return {
            "au12_baseline_pre": float(np.mean(baseline_obs)) if baseline_obs else None,
            "au12_lift_p90": None,
            "au12_lift_peak": None,
            "au12_peak_latency_ms": None,
        }

    baseline_pre = float(np.mean(baseline_obs)) if baseline_obs else 0.0
    values = np.array([o["intensity"] for o in measure_obs], dtype=float)
    p90_post = float(np.percentile(values, 90))
    peak_idx = int(np.argmax(values))
    peak_post = float(values[peak_idx])
    peak_latency_ms = float((measure_obs[peak_idx]["timestamp_s"] - stimulus_time_s) * 1000.0)

    return {
        "au12_baseline_pre": baseline_pre,
        "au12_lift_p90": max(0.0, p90_post - baseline_pre),
        "au12_lift_peak": max(0.0, peak_post - baseline_pre),
        "au12_peak_latency_ms": peak_latency_ms,
    }


def compute_peak_lagged_corr(x: np.ndarray, y: np.ndarray, max_lag: int) ->
↪  tuple[Optional[float], Optional[int]]:
    best_r: Optional[float] = None
    best_lag: Optional[int] = None
    for lag in range(0, max_lag + 1):
        if lag == 0:
            x_lag = x
            y_lag = y
        else:
            x_lag = x[:-lag]
            y_lag = y[lag:]
        if len(x_lag) < 4 or len(y_lag) < 4:
            continue
        if np.std(x_lag) == 0.0 or np.std(y_lag) == 0.0:
            continue
        r, _ = pearsonr(x_lag, y_lag)
        if best_r is None or float(r) > best_r:
            best_r = float(r)
            best_lag = lag
    return best_r, best_lag
```

SECTION 8

Deterministic delivery validation and response inference specification

Semantic evaluation is constrained so language understanding remains replayable and safe to persist. The active path emits bounded delivery-validation outputs and bounded response-inference fields, never free-form model `reasoning`. The `reasoning` field is a compatibility field for a bounded delivery reason code; response-related explanations live only in bounded `response_inference` fields.

Module D determines whether the selected `stimulus_attempt` was delivered under `expected_stimulus_rule` and, separately, whether observable evidence indicates that the `subject_response` satisfied `expected_response_rule`. Spoken greeting evaluation remains a supported compatibility case, but the canonical model is stimulus-response.

To perform robust delivery validation while maintaining reproducibility, LSIE-MLF uses a deterministic method registry. The primary method for spoken greeting and text-semantic rules is a local Cross-Encoder executed in `gpu_ml_worker`. An optional Azure OpenAI gray-band fallback may be invoked only when the calibrated primary score falls inside a configured ambiguity interval. Event-native stimuli use deterministic bounded platform-event evidence under the same output contract.

The top-level `is_match` boolean remains the §7B reward gate G_t in {0, 1}. It shall mean delivery validation only. When top-level `is_match` is True, *P90 AU12* intensity contributes to the posterior update; otherwise the reward is zero. The `confidence_score` value is the active method's delivery-validation probability estimate and remains analytical beyond the binary gate; it does not modulate the §7B reward path.

Subject response is represented only in the nested `response_inference` object. The `response_inference.is_match`, `registration_status`, `response_reason_code`, `matched_response_time`, and `evidence_span_ref` values are observational outputs and must not alter §7B reward computation or Thompson Sampling updates in v4.0.

The top-level `reasoning` field is a legacy compatibility field whose value must be one bounded delivery-validation reason code. It is not a free-form rationale. Response reason codes live only under `response_inference.response_reason_code`. Implementations must not persist free-form semantic rationales or free-form response rationales.

Response inference shall always emit an object. The `pending` state is non-terminal and valid only before `trigger_time_utc` + `response_observation_horizon_s` closes or before cross-segment evidence has arrived. The `observable_response`, `no_observable_response`, `ambiguous_response`, and `invalid_trigger` states are terminal.

Semantic shadow mode runs a candidate delivery-validation method in parallel with the active method and records its score for evaluation only. Shadow-mode outputs must not alter live top-level `is_match`, `confidence_score`, reward computation, response inference, or Thompson Sampling updates.

§8.2.1 Inference parameter constraints

PARAMETER	VALUE	JUSTIFICATION
`temperature`	`0.0`	Collapses the probability distribution to deterministic token selection.
`top_p`	`1.0`	Disables nucleus sampling ensuring deterministic token selection.
`max_tokens`	`500`	Ensures adequate space for gray-band fallback deliberation plus structured JSON output.
`response_format`	`JSON Schema Structured Outputs`	Forces the model to return machine-parseable JSON rather than conversational text.
`seed`	`42`	Activates Azure OpenAI deterministic fallback mode.
`cross_encoder_model_id`	`local://models/semantic/lsie-greeting-cross-encoder-v1`	Pins the primary local scorer artifact used for spoken-greeting and text-semantic delivery validation across deployments.
`cross_encoder_model_version`	`lsie-greeting-cross-encoder-v1.0.0`	Pins the exact primary scorer release used for deterministic delivery-validation replay and audit.
`match_threshold`	`0.72`	Defines the deterministic probability cutoff: scores >= 0.72 are matches outside the gray band.
`gray_band_interval`	`0.58 ⩽ score < 0.72`	Defines the only score interval in which llm_gray_band fallback may execute; score = 0.72 is reserved for the direct match threshold and is outside the gray band.
`cross_encoder_device_mode`	`cuda:0 deterministic when GPU is available; cpu deterministic fallback only when semantic_device_mode=cpu is configured`	Pins the execution device policy so primary scorer behavior is reproducible across deployments.
`semantic_calibration_version`	`semantic-greeting-calibration-v1.0.0`	Pins the calibration mapping from raw model scores to delivery-validation probability estimates.

§8.1 Deterministic semantic scorer registry

METHOD	ROLE / MODE	DETERMINISTIC	DESCRIPTION
`cross_encoder`	primary `local`	Yes	`cross_encoder` is the primary local semantic method for deterministic delivery validation. The pinned local model and calibration artifact allow replayed segments to reproduce the same bounded delivery score, and its output remains separate from observational response-inference fields.
`llm_gray_band`	fallback `azure_openai`	Yes	`llm_gray_band` is a constrained fallback rather than a general `reasoning` channel. It is invoked only for calibrated gray-band delivery-validation scores and returns the same bounded schema and reason-code vocabulary as the primary path.
`azure_llm_legacy`	legacy `azure_openai`	Yes	`azure_llm_legacy` exists for historical replay and compatibility records. Older semantic outputs can be interpreted through this method, but it is not the active v4 desktop scoring surface.

§8.3 Expected output schema

The delivery-validation output schema keeps semantic results bounded: a boolean match, a confidence score, deterministic method metadata, and a constrained reason code rather than free-form rationale text.

Canonical output schema details are retained in the embedded machine-readable artifact.

§8.4 System prompt (canonical)

The canonical system prompt instructs semantic evaluation to judge bounded delivery evidence without expanding the persisted output beyond the approved schema.

Canonical prompt text is retained in the embedded machine-readable artifact.

SECTION 9

Desktop runtime topology

LSIE-MLF v4.0 runs as a Windows-native desktop ProcessGraph launched by `python -m services.desktop_app`. Six local processes divide UI/API hosting, capture supervision, orchestration, ML inference, SQLite analytics writes, and cloud synchronization; IPC queues carry control envelopes

and shared-memory blocks carry large PCM windows. Heavy ML dependencies stay in `gpu_ml_worker`, SQLite writes stay in `analytics_state_worker`, and cloud availability is never required for local inference.

LSIE-MLF v4.0 runs as a Windows-native six-process desktop graph. The launcher hydrates a python-build-standalone runtime with `uv` and starts child processes with spawn semantics and freeze_support. Retired container, broker, and local database surfaces are outside the normative desktop runtime contract.

§9.1 Desktop process graph

PROCESS	ROLE	ALLOWED IMPORTS	STATE ACCESS
`ui_api_shell`	Hosts PySide6 operator console and localhost FastAPI API shell.	`PySide6, fastapi, uvicorn, pydantic`	Read-only SQLite through sqlite_reader with PRAGMA query_only=1.
`capture_supervisor`	Supervises scrcpy, ADB, and FFmpeg native subprocesses and emits capture/drift events.	`subprocess, pywin32, multiprocessing`	No direct SQLite writes; may read install manifest through parent-provided configuration.
`module_c_orchestrator`	Applies drift offsets, assembles 30-second segments, extracts AU12, attaches physiological context, captures bandit snapshot, and produces desktop IPC messages.	`mediapipe, numpy, av, multiprocessing.shared_memory`	Read-only experiment cache; no reward/posterior writes.
`gpu_ml_worker`	Consumes SharedMemory PCM windows and performs speech, semantic, acoustic, and ML inference.	`torch, ctranslate2, mediapipe, faster_whisper, sentence_transformers, transformers, parselmouth`	No direct SQLite writes.
`analytics_state_worker`	Sole SQLite writer. Computes reward, posterior updates, attribution, local metrics, and posterior deltas.	`sqlite3, pandas, scipy, pydantic, numpy`	Exclusive SQLite writer with WAL/NORMAL/busy_timeout/temp_store=MEMORY.
`cloud_sync_worker`	Synchronizes signed experiment bundles, sessions, telemetry, and posterior deltas with the cloud control plane.	`httpx, keyring, pydantic, cryptography`	Reads pending uploads and writes cloud sync acknowledgements through analytics_state_worker; stores secrets only in Windows Credential Manager.

§9.2 Desktop IPC contract

The desktop IPC contract separates lightweight control messages from shared-memory media payloads. Queue messages identify the shared-memory block, byte length, `checksum`, and lifecycle state, while consumers attach without taking ownership and dirty-state recovery removes orphaned blocks after crashes.

Control-message schema and dirty-state recovery details are retained in the embedded machine-readable artifact.

§9.3 Native subprocess supervision

Native subprocess supervision keeps unstable device-capture children inside a platform-owned process tree. On Windows, `ADB`, `scrcpy`, and `FFmpeg` are grouped under Win32 Job Objects so shutdown and crash recovery can reap the whole tree instead of leaving stale capture processes behind.

§9.4 Runtime base

The runtime base defines the desktop launcher environment rather than a container image. A `python-build-standalone` distribution hydrated through `uv` and frozen through a PyInstaller shell is the release boundary, with Windows 11 x64 as the tier-1 target.

PARAMETER	SPECIFICATION
Python distribution	python-build-standalone
Dependency manager	uv
Packaging shell	PyInstaller one-folder launcher shell only
Tier 1 Platform	Windows 11 x64

§9.5 Startup sequence

STEP	SERVICE	DEPENDS ON	READINESS GATE
1	`desktop_launcher`	—	python-build-standalone and uv.lock health check
2	`analytics_state_worker`	`desktop_launcher`	SQLite schema migration and WAL pragma verification
3	`ui_api_shell`	`analytics_state_-worker`	localhost FastAPI health and PySide6 shell shown
4	`cloud_sync_worker`	`analytics_state_-worker`	OAuth/PKCE token or offline-cache mode initialized

STEP	SERVICE	DEPENDS ON	READINESS GATE
5	capture_supervisor	analytics_state_-worker	ADB device and scrcpy preflight checks
6	module_c_-orchestrator	capture_supervisor, analytics_state_-worker	drift offset stream and segment assembler ready
7	gpu_ml_worker	module_c_orchestrator	ML import preflight and device resolver check

§9.6 Tier-2 platform design intent

Tier-2 platform notes preserve macOS design constraints without making them part of the v4.0 release target. They document expected substitutions for process supervision, credential storage, app nap suppression, and model-device routing so future v4.1 work has stable context without changing the Windows-native runtime contract.

SECTION 10

Dependency and runtime matrix

The dependency matrix records the packages and system requirements needed to reproduce the desktop runtime, ML path, and retained cloud/server surfaces. Install targets keep base desktop launch separate from ML-heavy packages and server/cloud dependencies. Package movement across those targets changes runtime boundaries and therefore belongs in review, not in incidental cleanup.

§10.1 System requirements

COMPONENT	REQUIRED VER	NOTES
Tier 1 host OS	Windows 11 x64	Production and developer Tier 1 host for v4.0.
Python runtime	3.11.x	Provided by python-build-standalone. Python 3.12+ is rejected because mediapipe == 0.10.9 has no supported PyPI wheel for this stack.
Runtime resolver	uv locked	uv.lock is checked in and must remain consistent with pinned legacy requirements.
Production NVIDIA GPU	Turing SM 7.5+	Required for production GPU speech path with torch 2.4.x and ctranslate2 >= 4.5.0.
Developer Pascal over-ride	LSIE_DEV_FORCE_CPU_SPEECH=1	Permits GTX 1080 Ti development by forcing speech tran-scription to CPU while leaving PyTorch semantic scoring and MediaPipe on GPU.
NVIDIA CUDA	12.x	Production CUDA runtime for torch and CTranslate2 stack.

COMPONENT	REQUIRED VER	NOTES
cuDNN	9.x	Required by torch 2.4.x stack. v3.4 cuDNN 8/Pascal production justification is retired.
scrcpy	3.3.4	Required for raw Android USB audio/video extraction; v3.3.4 provides –no-playback and dual-instance port range support.
SQLite	3.43+	Local derived-state store in WAL mode. The analytics_state_worker process is the sole writer.
Windows Credential Manager	OS-provided	Secret storage backend accessed through keyring.

§10.2 Python package pinning

PACKAGE	VERSION	INSTALL TARGET(S)	PURPOSE
faster-whisper	1.2.1	gpu_ml_worker	Speech transcription wrapper using CTranslate2 runtime; desktop PCM arrives through SharedMemory.
mediapipe	0.10.9	module_c_-orchestrator, gpu_ml_worker	Face Mesh 478 landmark inference; legacy mp.solutions.face_mesh API preserved.
praat-parselmouth	0.4.4	gpu_ml_worker	Praat acoustic feature extraction
spacy	3.7.x	gpu_ml_worker	NLP preprocessing pipeline
psycopg2-binary	2.9.x	cloud_api	Cloud PostgreSQL driver for the cloud control plane; not a local desktop persistence dependency.
pandas	2.2.x	analytics_state_worker	Time-series and analytics processing.
scipy	1.13.*	analytics_state_worker, gpu_ml_worker	Statistical computation and acoustic analytics.
celery	5.4.x (v3.4 LTS only)	retained_legacy_or_deferred_-integration	Retained legacy/server distributed task queue pin; not a normative v4.0 desktop runtime dependency.
redis	5.x (v3.4 LTS only)	retained_legacy_or_deferred_-integration	Retained legacy/server broker pin; not a normative v4.0 desktop runtime dependency.
fastapi	0.110.x	ui_api_shell, cloud_api	Localhost desktop API shell and cloud API service framework.
uvicorn	0.29.x	ui_api_shell, cloud_api	ASGI server runtime for local and cloud FastAPI surfaces.
pydantic	2.x	all_processes, cloud_api	Schema validation and schema-on-write privacy enforcement.

PACKAGE	VERSION	INSTALL TARGET(S)	PURPOSE
numpy	1.26.x	module_c_-orchestrator, gpu_ml_worker, analytics_state_worker	Vectorized AU12 calculations and ML preprocessing.
pycryptodome	⩾3.20.0	retained_legacy_or_deferred_-integration	Retained AES-256-GCM utility pin for historical Ephemeral Vault surfaces; active v4 desktop forbids raw-media persistence.
patchright	1.58.2	retained_legacy_or_deferred_-integration	Deferred context-enrichment browser automation pin; not imported by the active desktop ProcessGraph.
TikTokLive	5.0.x	retained_legacy_or_deferred_-integration	Retained/deferred live platform ingestion pin; not required for launching the active desktop ProcessGraph.
PySide6	⩾6.11	ui_api_shell	Native Qt operator console hosted by the desktop UI/API shell.
torch	2.4.x	gpu_ml_worker	Local Cross-Encoder semantic scoring runtime and GPU tensor backend.
transformers	4.46.x	gpu_ml_worker	Transformer runtime for deterministic local semantic scorer
sentence-transformers	3.2.x	gpu_ml_worker	Cross-Encoder semantic similarity models
ctranslate2	⩾4.5.0	gpu_ml_worker	CTranslate2 runtime used by faster-whisper production GPU speech path with cuDNN 9-compatible stack.
pywin32	pinned in uv.lock	capture_supervisor	Win32 Job Objects for native subprocess cleanup.
keyring	pinned in uv.lock	ui_api_shell, cloud_sync_worker	Windows Credential Manager backed secret storage.
httpx	pinned in uv.lock	desktop_launcher, cloud_sync_worker	Runtime download, resumable first-run install, and HTTPS cloud sync.
cryptography	pinned in uv.lock	cloud_sync_-worker, cloud_api	Ed25519 ExperimentBundle signature verification and cloud auth support.
sqlite-utils	optional	developer_tools	Developer inspection utilities only; runtime uses Python sqlite3.

SECTION 11

Variable extraction matrix

The variable matrix links runtime events to stored analytics. Each row identifies where a value is produced, the unit or range that makes it interpretable, and the destination used for replay, dashboards,

attribution, or synchronization. Reward, attribution, co-modulation, acoustic analysis, and cloud telemetry fields all need a traceable producer and a bounded representation.

This matrix enumerates variables produced or consumed by LSIE-MLF v4.0. Canonical stimulus-response names are used for new fields: `stimulus_attempt`, `trigger_time_utc`, `delivery_validation`, `subject_response`, `response_observation_horizon_s`, and `outcome_event`. Legacy greeting-named fields are compatibility aliases only and must be derived from canonical stimulus-response rules when both exist.

VARIABLE PROFILE	EXTRACTION & ROUTING
Encoded Video Stream `byte stream (H.264 in MKV container)` Module A → Transient (kernel buffer)	scrcpy USB mirroring; H.264 encoded stream in MKV container via `/dev/bus/usb` passthrough.
Raw Audio PCM `byte stream (s16le)` Unit/range: 48 kHz mono Module A → Transient IPC pipe buffer	scrcpy raw audio capture; s16le 48 kHz mono to IPC Pipe.
Live Event Payloads `dict` Module B → Transient deque buffer	TikTokLive WebSocket ingestion; parsed protobuf event dictionaries.
Action Combo Trigger `bool` Unit/range: True/False Module B → Persistent Store events table	Composite temporal event correlation across concurrent chat actions.
UTC Drift Offset `float` Unit/range: seconds Module C → Transient variable	ADB epoch polling every 30s; `drift_offset` = host UTC − Android epoch.
Resampled Audio Chunks `byte stream (s16le)` Unit/range: 16 kHz mono Module C → Transient stdout pipe	FFmpeg resampling subprocess; 48 kHz → 16 kHz mono s16le on `pipe:1`.
3D Facial Landmarks `np.ndarray (478,3)` Unit/range: normalized coordinates Module C → Transient memory	MediaPipe Face Mesh in Module C; 478-vertex 3D landmark array per sampled frame.
AU12 Intensity Score `float` Unit/range: 0.0–5.0 (compute_intensity) or 0.0–1.0 (compute_bounded_intensity for Thompson Sampling reward pipeline) Module C → Persistent Store metrics table	AU12Normalizer (§7A) executed in Module C; IOD-normalized lip-corner distance with neutral baseline subtraction. Exposed as 0.0–5.0 diagnostic scale and 0.0–1.0 bounded reward scale.

VARIABLE PROFILE	EXTRACTION & ROUTING
Vocal Pitch F0 `float` Unit/range: 75–600 Hz Module D → Persistent Store metrics table	Legacy segment-level compatibility variable (deprecated as a canonical acoustic output in v3.3). Praat via parselmouth; fundamental frequency extraction from speech audio. Downstream consumers must migrate to the windowed §7D F0 fields.
Jitter `float` Unit/range: 0–1 ratio Module D → Persistent Store metrics table	Legacy segment-level compatibility variable (deprecated as a canonical acoustic output in v3.3). Praat via parselmouth; cycle-to-cycle pitch perturbation ratio. Downstream consumers must migrate to the windowed §7D jitter fields.
Shimmer `float` Unit/range: 0–1 ratio Module D → Persistent Store metrics table	Legacy segment-level compatibility variable (deprecated as a canonical acoustic output in v3.3). Praat via parselmouth; cycle-to-cycle amplitude perturbation ratio. Downstream consumers must migrate to the windowed §7D shimmer fields.
ASR Transcription `string` Unit/range: UTF-8 text Module D → Persistent Store transcripts table	faster-whisper with CTranslate2 INT8 backend; segment-level speech-to-text.
Delivery Validation Result `JSON object` Unit/range: confidence [0.0, 1.0]; is_match True/False; reasoning bounded enum Module D → Persistent Store evaluation table	Deterministic delivery-validation contract (§8); semantic scoring or bounded event evidence produces top-level legacy fields `reasoning`, `is_match`, and `confidence_score`. The top-level `reasoning` value is a bounded delivery reason code, not free-form rationale. The top-level `is_match` remains the §7B reward gate and is separate from `response_inference`.
Evaluation Variance `float` Unit/range: 0–infinity Module E → Persistent Store experiments table	Thompson Sampling posterior statistics; variance of Beta distribution per arm.
External Context Metadata `JSON dict` Module F → Persistent Store context table	patchright headless browser scraping; creator and trending topic metadata.
gated_reward `float` Unit/range: [0.0, 1.0] Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); r_t = `P90` × `delivery_validation_gate`. Response-inference outputs are observational and do not update the posterior.
p90_intensity `float` Unit/range: [0.0, 1.0] Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); 90th percentile of bounded AU12 in stimulus window.

VARIABLE PROFILE	EXTRACTION & ROUTING
delivery_validation_gate `int` Unit/range: {0, 1} Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); binary gate from top-level delivery-validation `is_match`: 1 if True, 0 otherwise. It is not derived from `response_-inference`.
n_frames_in_window `int` Unit/range: [0, +infinity) Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); count of AU12 observations in stimulus window.
au12_baseline_pre `float | null` Unit/range: [0.0, +infinity) or null Module E → Phase3_EncounterLog JSON payload and Persistent Store attribution_score table	`compute_reward()` (§7B) and §7E attribution diagnostics; mean bounded AU12 in the pre-stimulus window, or null.
stimulus_time `float` Unit/range: UTC epoch seconds Module C → Transient variable	Drift-corrected UTC epoch of stimulus trigger_time; defines baseline and measurement windows for `compute_reward()` (§7B).
RMSSD (Streamer) `float | null` Unit/range: milliseconds, >= 0 or null Module C → Persistent Store physiology_log table	Derived at segment assembly from normalized physiological chunks buffered in Module C (§4.B.2, §4.C.4). Uses a trailing 5-minute window with ibi precedence over session. Forced to null when `validity_ratio < PHYSIO_VALIDITY_MIN`.
RMSSD (Operator) `float | null` Unit/range: milliseconds, >= 0 or null Module C → Persistent Store physiology_log table	Derived at segment assembly from normalized physiological chunks buffered in Module C (§4.B.2, §4.C.4). Uses a trailing 5-minute window with ibi precedence over session. Forced to null when `validity_ratio < PHYSIO_VALIDITY_MIN`.
Heart Rate (Streamer/Operator) `int | null` Unit/range: 20–300 BPM or null Module C → Persistent Store physiology_log table	Rounded mean of overlapping `heart_rate_items_bpm` values in the trailing derivation window (§4.C.4); null when no heart-rate values exist.
Physiological Validity Ratio `float` Unit/range: [0.0, 1.0] Module C → Persistent Store physiology_log table	`validity_ratio = sum(valid_sample_count) / max(sum(expected_sample_count), 1)` over the trailing derivation window (§4.C.4).
Physiological Validity Flag `bool` Unit/range: True / False Module C → Persistent Store physiology_log table	`is_valid = (validity_ratio ⩾ PHYSIO_VALIDITY_MIN)`; default threshold 0.80 (§4.C.4).

VARIABLE PROFILE	EXTRACTION & ROUTING
Physiological Source Kind `string` Unit/range: {'ibi', 'session'} Module C → Persistent Store physiology_log table	ibi when the snapshot is derived from interbeat-interval chunks; otherwise session when derived from provider-session HRV chunks (§4.C.4).
Physiological Derivation Method `string` Unit/range: {'provider', 'server'} Module C → Persistent Store physiology_log table	`server` for IBI-derived RMSSD; `provider` for provider-session HRV snapshots unless later recomputed (§4.C.4).
Physiological Window Length `int` Unit/range: seconds, > 0 Module C → Persistent Store physiology_log table	Length of the trailing derivation window; default `PHYSIO_DERIVE_WINDOW_S = 300` (§4.C.4).
Physiological Freshness `float` Unit/range: seconds, >= 0 Module C → Persistent Store physiology_log table	Computed at segment assembly: `wall_clock - source_timestamp_utc` (§4.C.4).
Physiological Staleness Flag `bool` Unit/range: True / False Module C → Persistent Store physiology_log table	`freshness_s > PHYSIO_STALENESS_THRESHOLD_S` (default 600s) (§4.C.4).
Co-Modulation Index `float | null` Unit/range: [-1.0, +1.0] or null Module E → Persistent Store co-modulation_log table	Pearson correlation on 1-minute resampled, time-aligned valid streamer/operator RMSSD sequences over a 10-minute rolling window (§7C).
Physiological Coverage Ratio `float` Unit/range: [0.0, 1.0] Module E → Persistent Store co-modulation_log table	Fraction of 1-minute analysis bins with valid paired RMSSD observations over the 10-minute co-modulation window (§7C).
F0 Validity (Measure Window) `bool` Unit/range: True / False Module D → Persistent Store metrics table	Observational acoustic validity gating (§7D); true only when W_measure satisfies voiced-coverage and stable-island thresholds.
F0 Validity (Baseline Window) `bool` Unit/range: True / False Module D → Persistent Store metrics table	Observational acoustic validity gating (§7D); true only when W_baseline satisfies voiced-coverage and stable-island thresholds.

VARIABLE PROFILE	EXTRACTION & ROUTING
Perturbation Validity (Measure Window) `bool` Unit/range: True / False Module D → Persistent Store metrics table	Perturbation-span validity gating (§7D); true only when W_measure contains an eligible perturbation span.
Perturbation Validity (Baseline Window) `bool` Unit/range: True / False Module D → Persistent Store metrics table	Perturbation-span validity gating (§7D); true only when W_baseline contains an eligible perturbation span.
Voiced Coverage (Measure Window) `float` Unit/range: >= 0 seconds Module D → Persistent Store metrics table	Cumulative voiced coverage in W_measure computed under §7D voiced-only masking.
Voiced Coverage (Baseline Window) `float` Unit/range: >= 0 seconds Module D → Persistent Store metrics table	Cumulative voiced coverage in W_baseline computed under §7D voiced-only masking.
F0 Mean (Measure Window) `float | null` Unit/range: 75–600 Hz or null Module D → Persistent Store metrics table	Mean voiced-only F0 in W_measure under §7D; null when f0_valid_measure is false.
F0 Mean (Baseline Window) `float | null` Unit/range: 75–600 Hz or null Module D → Persistent Store metrics table	Mean voiced-only F0 in W_baseline under §7D; null when f0_valid_baseline is false.
F0 Delta (Semitones) `float | null` Unit/range: real-valued semitone difference or null Module D → Persistent Store metrics table	12 * log2(f0_mean_measure_hz / f0_mean_baseline_hz) under §7D; null when either dependent F0 mean is null or baseline F0 is non-positive.
Jitter Mean (Measure Window) `float | null` Unit/range: 0–1 ratio or null Module D → Persistent Store metrics table	Praat local jitter computed on the selected perturbation span in W_measure under §7D; null when perturbation_valid_measure is false.

VARIABLE PROFILE	EXTRACTION & ROUTING
Jitter Mean (Baseline Window) `float | null` Unit/range: 0–1 ratio or null Module D → Persistent Store metrics table	Praat local jitter computed on the selected perturbation span in W_baseline under §7D; null when perturbation_valid_baseline is false.
Jitter Delta `float | null` Unit/range: real-valued difference or null Module D → Persistent Store metrics table	jitter_mean_measure - jitter_mean_baseline under §7D; null when either dependent jitter mean is null.
Shimmer Mean (Measure Window) `float | null` Unit/range: 0–1 ratio or null Module D → Persistent Store metrics table	Praat local shimmer computed on the selected perturbation span in W_measure under §7D; null when perturbation_valid_measure is false.
Shimmer Mean (Baseline Window) `float | null` Unit/range: 0–1 ratio or null Module D → Persistent Store metrics table	Praat local shimmer computed on the selected perturbation span in W_baseline under §7D; null when perturbation_valid_baseline is false.
Shimmer Delta `float | null` Unit/range: real-valued difference or null Module D → Persistent Store metrics table	shimmer_mean_measure - shimmer_mean_baseline under §7D; null when either dependent shimmer mean is null.
Segment ID `string` Unit/range: 64-character lowercase hex Module C → InferenceHandoffPayload and Persistent Store attribution_event table	Deterministic SHA-256 over stable segment identity fields (`session_id`, `segment_window_start_utc`, `segment_window_end_utc`) in Module C (§6.1, §7E).
Bandit Decision Snapshot `JSON object` Module C → InferenceHandoffPayload and Persistent Store attribution_event table	Captured in Module C immediately after Thompson Sampling arm selection and before any posterior update; includes selected arm, candidate arms, posterior parameters, sampled thetas when available, compatibility expected_greeting, and stimulus-response rule metadata when present.
semantic_p_match `float` Unit/range: [0.0, 1.0] Module D → Persistent Store evaluation table and attribution_score table	Deterministic delivery-validation probability estimate produced by the active §8 method; primary local Cross-Encoder for semantic stimuli, with optional gray-band fallback where configured.

VARIABLE PROFILE	EXTRACTION & ROUTING
semantic_reason_code `string` Unit/range: bounded enum Module D → Persistent Store evaluation table	Bounded delivery-validation route/result code emitted by the deterministic §8 contract; response reason codes are carried only inside `response_inference`.
soft_reward_candidate `float | null` Unit/range: [0.0, 1.0] or null Module E → Persistent Store attribution_score table	`soft_reward_candidate = semantic_p_match * proxy_reward` under §7E observational attribution only. It remains tied to delivery-validation scoring and does not incorporate `response_inference` unless a later reward-path version allows it.
au12_lift_p90 `float | null` Unit/range: [0.0, 1.0] or null Module E → Persistent Store attribution_score table	`max(0, P90_post - au12_baseline_pre)` under §7E.
au12_lift_peak `float | null` Unit/range: [0.0, 1.0] or null Module E → Persistent Store attribution_score table	`max(0, Pmax_post - au12_baseline_pre)` under §7E.
au12_peak_latency_ms `float | null` Unit/range: milliseconds or null Module E → Persistent Store attribution_score table	Latency from stimulus trigger timestamp to the peak AU12 observation in the post-stimulus measurement window under §7E.
sync_peak_corr `float | null` Unit/range: [-1.0, 1.0] or null Module E → Persistent Store attribution_score table	Maximum lagged Pearson correlation over the configured lag range under §7E lag-aware synchrony analytics.
sync_peak_lag `int | null` Unit/range: lag units or null Module E → Persistent Store attribution_score table	Lag value associated with `sync_peak_corr` under §7E lag-aware synchrony analytics.
outcome_link_lag_s `float` Unit/range: seconds, >= 0 Module E → Persistent Store event_outcome_link table	Deterministic event→outcome lag in seconds stored on EventOutcomeLink records under §7E.
attribution_finality `string` Unit/range: {'online_provisional', 'offline_final'} Module E → Persistent Store attribution tables	Lifecycle state assigned by Module E attribution finalization jobs under §7E.

VARIABLE PROFILE	EXTRACTION & ROUTING
SharedMemory Audio Segment Name `string` Unit/range: lsie_ipc_pcm_* Module C → Queue control message only	Generated by module_c_orchestrator when staging a 30-second PCM window in SharedMemory.
SharedMemory Byte Length `integer` Unit: bytes Range: >0 Module C → Queue control message only	Measured from the exact PCM byte window staged for gpu_ml_worker.
SharedMemory Checksum `hex string` Range: SHA-256 Module C → Queue control message and diagnostics	SHA-256 over staged PCM bytes before SharedMemory handoff.
Bandit Random Seed `unsigned integer` Range: 0..18446744073709551615 Module C → BanditDecisionSnapshot	First 8 bytes of SHA-256(session_id || segment_window_start_utc || stimulus_time || policy_version), interpreted big-endian; `_stimulus_time` is the trigger_time compatibility field.
Posterior Delta `object` Range: 0≤delta≤1 Module E → SQLite posterior_delta_log and cloud PostgreSQL	analytics_state_worker computes fractional Beta-Bernoulli update and emits idempotent delta.
Cloud Event ID `UUIDv5 string` Unit/range: UUID Module F → SQLite pending_uploads and cloud posterior_delta_log	UUIDv5 over client_id, segment_id, experiment_id, and arm_id for exactly-once sync.
stimulus_id `UUID string or null` Unit/range: UUID or null Module C → InferenceHandoffPayload, attribution_event table, and derived telemetry uploads	Attached in Module C from normalized stimulus-trigger metadata and/or active-arm configuration; identifies the selected stimulus attempt when available.
stimulus_modality `string or null` Unit/range: bounded enum or null Module C → InferenceHandoffPayload, BanditDecisionSnapshot, attribution_event table, and derived telemetry uploads	Attached in Module C from normalized stimulus-trigger metadata and/or active-arm configuration.

VARIABLE PROFILE	EXTRACTION & ROUTING
stimulus_payload `JSON object or null` Unit/range: bounded object or null; no raw media or unbounded content Module C → InferenceHandoffPayload and BanditDecisionSnapshot; derived-only telemetry when allowed by cloud schemas	Modality-specific bounded stimulus content or trigger metadata copied from the selected active arm or normalized trigger event after privacy filtering. The payload must use modality-specific allowed keys, finite string lengths, and no raw media, provider bodies, or free-form rationale fields.
expected_stimulus_rule `string or null` Unit/range: bounded rule text or null Module C → InferenceHandoffPayload and BanditDecisionSnapshot	Bounded rule from the selected active arm describing what counts as intended stimulus delivery. `_expected_greeting` is a replay compatibility alias derived from this canonical field when both are present.
expected_response_rule `string or null` Unit/range: bounded rule text or null Module C → InferenceHandoffPayload and BanditDecisionSnapshot	Bounded rule from the selected active arm describing qualifying observable subject_response evidence for `response_inference`; it never changes the top-level delivery_validation gate.
response_observation_horizon_s `float or null` Unit: seconds Range: 0..30 seconds or null only when no selected stimulus_attempt exists Module C → InferenceHandoffPayload and derived analytics telemetry	Maximum post-trigger observation horizon selected by the active arm for response inference. The interval is [trigger_time_utc, trigger_time_utc + horizon]; pending is valid only before the horizon closes or while cross-segment evidence is unavailable.
response_inference.is_match `bool` Unit/range: True/False Module D → InferenceHandoffPayload response_inference object, attribution_event table, and derived telemetry uploads	Module D response-inference contract; true only when qualifying observable response evidence satisfies `expected_response_rule` within the configured horizon.
response_inference.confidence_score `float` Unit/range: [0.0, 1.0] Module D → InferenceHandoffPayload response_inference object and derived telemetry uploads	Bounded confidence emitted by Module D for the response-inference result; observational only in v4.0 final.

VARIABLE PROFILE	EXTRACTION & ROUTING
response_inference.registration_status `string` Range: observable_response | no_observable_response | ambiguous_response | invalid_trigger | pending Module D → InferenceHandoffPayload response_inference object, attribution_event table, and derived telemetry uploads	Bounded Module D response registration state. The pending state is non-terminal and legal only before horizon close; observable_response, no_observable_response, ambiguous_response, and invalid_trigger are terminal.
response_inference.response_reason_code `string` Unit/range: bounded enum Module D → InferenceHandoffPayload response_inference object and derived telemetry uploads	Bounded Module D reason code for response inference. It must not be persisted as a free-form rationale and must not replace top-level `reasoning`.
response_inference.matched_response_time `float or null` Unit: seconds Module D → InferenceHandoffPayload response_inference object, attribution_event table, and observational analytics	Module D timestamp for the first qualifying observable subject_-response, expressed relative to the same UTC epoch basis as trigger_time_utc. It is non-null only when registration_status = observable_response.

SECTION 12

Error handling and recovery specification

Error handling is deterministic degradation rather than best-effort recovery. Capture loss, device loss, worker crashes, queue overload, semantic ambiguity, acoustic insufficiency, database contention, and cloud outages produce bounded states that can be observed and replayed without changing reward semantics. Recovery may preserve liveness, but it cannot fabricate valid analytics, bypass schema validation, or promote observational response signals into reward updates.

LSIE-MLF v4.0 defines deterministic recovery behavior over the desktop process graph. Container restarts and broker retries are replaced by child-process supervision, Queue backpressure, Shared-Memory cleanup, SQLite WAL recovery, and cloud-sync backlog handling.

Hardware device loss

MODULE	BEHAVIOR
capture_supervisor	USB reconnection polling every 2 seconds for 60 seconds; Win32 Job Object terminates stale scrcpy/ADB/FFmpeg children before revival.

Worker process crash

MODULE	BEHAVIOR
capture_supervisor	Parent process reaps native children through Job Object and emits deterministic capture failure event.

Temporal drift fault

MODULE	BEHAVIOR
module_c_-orchestrator	ADB drift freezes after 3 failures and resets after 300 seconds; corrected_ts = original_ts + last_valid_offset while frozen.

SharedMemory leak

MODULE	BEHAVIOR
module_c_-orchestrator	Dirty State Recovery unlinks orphaned lsie_ipc_* blocks at startup; owner zeroizes and unlinks after gpu_ml_worker acknowledgement.

ML import failure

MODULE	BEHAVIOR
gpu_ml_worker	Preflight fails process readiness if torch/ctranslate2/mediapipe/faster_whisper cannot co-load on a production Turing-or-newer GPU.

Pascal speech incompatibility

MODULE	BEHAVIOR
gpu_ml_worker	Developer machines may set LSIE_DEV_FORCE_CPU_SPEECH=1; production must use Turing SM 7.5+ GPU speech path.

Acoustic data quality

MODULE	BEHAVIOR
gpu_ml_worker	Insufficient voiced coverage, invalid perturbation spans, or Praat extraction failure produces deterministic false validity flags and null dependent acoustic outputs.

SQLite Busy/Crash recovery

MODULE	BEHAVIOR
analytics_state_-worker	Single writer uses WAL, synchronous=NORMAL, busy_timeout=5000, temp_-store=MEMORY; restart verifies schema, checkpoints/truncates WAL if required, and resumes pending uploads.

Reward bounds fault

MODULE	BEHAVIOR
analytics_state_-worker	Rewards outside [0,1] are rejected before posterior update; Gate 0 replay verifies r_t = P90 x G_t and fractional Beta-Bernoulli update.

Network disconnection

MODULE	BEHAVIOR
cloud_sync_worker	Enter offline mode; local inference and posterior updates continue; pending_uploads retry with backoff.

Cloud duplicate

MODULE	BEHAVIOR
cloud_sync_worker	Posterior deltas use UNIQUE(segment_id, client_id, arm_id) and INSERT … ON CONFLICT DO NOTHING; cloud posterior UPDATE occurs only on successful insert.

Privacy perimeter violation

MODULE	BEHAVIOR
cloud API	extra=forbid schemas reject raw PCM, decoded frames, voiceprints, raw provider bodies, complete physiological chunks, free-form semantic rationales, free-form response rationales, and unbounded stimulus payload content.

Operator UI exit

MODULE	BEHAVIOR
ui_api_shell	Parent shutdown closes child process graph and Job Objects; local SQLite remains consistent through WAL recovery.

SECTION 13

Autonomous implementation audit checklist

The audit checklist condenses the v4 desktop contract into reviewable invariants. Topology, IPC, privacy, deterministic replay, stimulus-response analytics, cloud sync, and hardware requirements remain visible as concrete checks after code changes. Each row is a prompt for evidence in tests, fixtures, or code references.

Mandatory v4.0 implementation controls gate workstreams before merge. Relevant Gate 0 and runtime-topology controls must pass, and stimulus-response checks use `delivery_validation` for reward gating and `response_inference` only for observational `subject_response` analytics.

#	AUDIT ITEM	VERIFICATION CRITERION
1	Runtime topology	Exactly six v4.0 desktop processes are defined: ui_api_shell, capture_supervisor, module_c_orchestrator, gpu_ml_worker, analytics_state_worker, cloud_sync_worker.
2	Import discipline	Only gpu_ml_worker imports the complete heavy ML set torch, ctranslate2, mediapipe, and faster_whisper; module_c_orchestrator may import MediaPipe only for AU12/Face Mesh.
3	Embedded runtime	python-build-standalone and uv.lock hydrate the desktop runtime; PyInstaller is used only for the launcher shell.
4	Windows subprocess supervision	capture_supervisor reaps ADB, scrcpy, and FFmpeg through Win32 Job Objects.
5	SharedMemory IPC	30-second PCM windows travel through SharedMemory; Queue control messages contain shared_memory_name, byte_length, and checksum, not _audio_data.

#	AUDIT ITEM	VERIFICATION CRITERION
6	SharedMemory cleanup	Consumers attach with track=False; Dirty State Recovery unlinks lsie_ipc_* orphaned blocks; audio blocks are zeroized before unlink.
7	Gate 0 corpus	Gate 0 replay corpus exists, includes stimulus-response fixture families, and blocks all workstreams.
8	Deterministic segment identity	segment_id = SHA256(session_id || canonical window_start || canonical window_end) and 30-second segment windows are preserved.
9	ADB drift contract	ADB drift freezes after 3 failures and resets after 300 seconds.
10	Reward pipeline	Reward pipeline implements §7B unchanged: P90 aggregation, binary delivery-validation gate from top-level is_match, fractional Beta-Bernoulli posterior updates with Beta(1,1) prior, and stimulus window boundaries [t+0.5s, t+5.0s].
11	Bandit snapshot determinism	BanditDecisionSnapshot.random_seed and decision_context_hash are required, captured before posterior update, and preserve canonical stimulus-response rule metadata when present.
12	SQLite local state	SQLite uses WAL/NORMAL/busy_timeout/temp_store=MEMORY and analytics_state_worker is the sole writer.
13	Privacy perimeter	No raw media, hydrated provider bodies, complete physiological chunks, voiceprints, unbounded stimulus payload content, free-form semantic rationales, or free-form response rationales persist locally or in cloud.
14	Cloud OAuth	Cloud sync uses OAuth 2.0 Authorization Code with PKCE; refresh tokens are stored only in Windows Credential Manager via keyring.
15	Signed ExperimentBundle	Experiment bundles are Ed25519-signed and cached locally only after signature verification.
16	Posterior delta exactly-once	Posterior deltas are idempotent through UNIQUE(segment_id, client_id, arm_id) and INSERT … ON CONFLICT DO NOTHING.
17	Production hardware floor	Production GPU path is validated on Turing SM 7.5+ with CUDA 12.x and cuDNN 9.x.
18	Pascal developer override	Pascal development requires LSIE_DEV_FORCE_CPU_SPEECH=1 for CPU speech transcription while preserving other GPU paths.
19	macOS Tier 2 deferral	macOS App Nap, Keychain, process-group, Hardened Runtime, and MPS routing are documented as Tier 2 v4.1 design intent only.
20	Stimulus-response fixture coverage	Conformance fixtures cover spoken-greeting and written-comment stimulus_attempts across delivered, no_observable_response, ambiguous_response, invalid_trigger, pending horizon, cross-segment finalization, and response-locked analytics cases.
21	Delivery-validation and response-inference separation	Top-level reasoning, is_match, and confidence_score remain the delivery_validation contract and §7B reward gate; response_inference is always a bounded object, uses pending only as a non-terminal state before horizon close, and remains observational in v4.0 final.
22	Stimulus traceability fields	Variable traceability covers stimulus_id, stimulus_modality, bounded stimulus_payload, expected_stimulus_rule, expected_response_rule, response_observation_horizon_s, and response_inference state fields without changing raw-media persistence boundaries.

SECTION 14

Cloud control plane and telemetry sync

The cloud control plane extends the desktop runtime without becoming a dependency for local operation. It receives bounded segment telemetry, idempotent posterior deltas, session lifecycle records, and signed experiment bundles behind OAuth/PKCE and schema-on-write privacy checks. Cloud sync may enrich coordination and review, but it must never require raw media upload or block local segment processing.

V4 separates desktop local execution from cloud-canonical experiment control. The cloud accepts only derived telemetry, including bounded stimulus-response analytics, through schema-on-write API models and distributes signed experiment bundles to the desktop cache.

§14.1 Cloud route registry

ROUTE	AUTH	PURPOSE AND BOUNDARY
`POST /v4/telemetry/segments`	Required	POST /v4/telemetry/segments accepts bounded derived segment telemetry from the desktop outbox. The route records what the local runtime already derived and rejects raw-media upload material at the schema boundary. Raw PCM, decoded frames, voiceprints, provider bodies, and free-form rationales stay inside the desktop privacy perimeter.
`POST /v4/telemetry/posterior_deltas`	Required	POST /v4/telemetry/posterior_deltas applies idempotent Thompson Sampling posterior changes computed locally by `analytics_state_worker`. Cloud state catches up to offline desktop decisions through unique segment, client, and arm keys rather than by replaying duplicate updates.
`GET /v4/experiments/bundle`	Required	GET /v4/experiments/bundle delivers signed experiment policy for local cache refresh. The desktop runtime can continue from a valid cached bundle while offline, and newly downloaded bundles become active only after signature verification.
`POST /v4/sessions`	Required	POST /v4/sessions creates the cloud representation of a locally managed session. The record coordinates review and synchronization metadata without becoming the source of truth for live desktop processing, and its creation path must tolerate retried outbox rows.

ROUTE	AUTH	PURPOSE AND BOUNDARY
`POST /v4/sessions/{id}/end`	Required	POST /v4/sessions/{id}/end closes the cloud session record after local work has ended. Offline operation can delay final sync, so late or repeated close messages must not become prerequisites for raw transient-media cleanup or SQLite finalization.
`POST /v4/auth/oauth/token`	Not required	POST /v4/auth/oauth/token performs OAuth 2.0 Authorization Code with PKCE token exchange and refresh. Token issuance is intentionally unauthenticated before credentials exist, but the request remains schema-bound and rate-limited; refresh tokens belong in Windows Credential Manager through `keyring`, not in SQLite tables or log files.

— END OF SPECIFICATION —